

Contents

BASIC COMMANDS:.....	1
ROOT DIRECTORY & HOME DIRECTORY:	6
NANO TEXT EDITOR:	8
REMOVE , RENAME AND MOVE FILES AND FOLDERS:	8
CAT COMMAND:	12
HEAD , TAIL COMMAND:	15
LINUX PERMISSIONS:	19
FILE MANAGEMENT:	28
MAKEFILE:	36
SHELL SCRIPTING.....	46
BASICS	52
Practise	83
SYSTEM CALLS:	87
PRACTISE QUESTIONS	95
IPC	126
ORDINARY PIPES.....	126
NAMED PIPES.....	133
Shared Pipe.....	141
EXAM TASKS	149
IPC SHARED	152
System Calls.....	162

BASIC COMMANDS:

1. Run the following commands:

Command	Description
`pwd`	Display the current working directory.
`ls`	List files and directories.
`cd Desktop`	Change the current directory to Desktop.
`mkdir NewFolder`	Create a new directory named "NewFolder".
`rmdir NewFolder`	Remove the empty directory "NewFolder".
`cp file.txt /tmp`	Copy "file.txt" to the "/tmp" directory.

We delete empty directory using rmdir folder name

We copy file to another folder using cp file.txt /tmp

We rename file using mv file.txt file1.txt

<code>`rm -r NewFolder`</code>	Remove the empty directory "NewFolder".
<code>`cp file.txt /tmp`</code>	Copy "file.txt" to the "/tmp" directory.
<code>`mv file.txt newname.txt`</code>	Rename "file.txt" to "newname.txt".
<code>`rm oldfile.txt`</code>	Remove the file "oldfile.txt".
<code>`touch newfile.txt`</code>	Create an empty file named "newfile.txt".
<code>`nano myfile.txt`</code>	Open the nano text editor for editing.

<code>`touch newfile.txt`</code>	Create an empty file named "newfile.txt".
<code>`nano myfile.txt`</code>	Open the nano text editor for editing.
<code>`cat myfile.txt`</code>	Display the content of "myfile.txt".
<code>`echo "Hello, Linux!"`</code>	Display the message "Hello, Linux!".
<code>`man ls`</code>	Display the manual page for `ls`.

<code>`chmod 755 script.sh`</code>	Change the permissions of "script.sh".
<code>`chown user:group myfile.txt`</code>	Change the owner and group of "myfile.txt".
<code>`ps aux`</code>	Display information about active processes.
<code>`kill 1234`</code>	Terminate the process with PID 1234.
<code>`top`</code>	Display real-time system statistics.
<code>`df -h`</code>	Display disk space usage.
<code>`du -h`</code>	Display file and directory space usage.

To search for a pattern in a file , do

Grep "Pattern" file.txt

If we want to search .txt file in certain folder

<code>`du -h`</code>	Display file and directory space usage.
<code>`grep "pattern" file.txt`</code>	Search for a pattern in "file.txt".
<code>`find /home/user -name "*.txt"`</code>	Search for ".txt" files in /home/user.
<code>`tar -czvf archive.tar.gz folder/`</code>	Create a compressed archive of "folder/".
<code>`gzip file.txt`</code>	Compress "file.txt" and replace it with "file.txt.gz".

- Pwd – print working directory

```
ooo@UBU-PC:~$ pwd
/home/ooo
ooo@UBU-PC:~$
```

-

- I want to see that in this directory , the folder I am on usme kia kia hai

```
ooo@UBU-PC:~$ pwd
/home/ooo
ooo@UBU-PC:~$ ls
'2023-10-08 15-50-29.mp4' '2023-10-16 18-13-35.mp4' Desktop snap
'2023-10-08 15-57-18.mp4' '2023-10-17 13-32-42.mp4' Documents Templates
'2023-10-08 16-03-47.mp4' '2023-10-17 13-33-14.mp4' Downloads Videos
'2023-10-10 19-07-37.mp4' '2023-10-17 13-35-00.mp4' Music 'VirtualBox VMs'
'2023-10-12 19-35-14.mp4' '2023-10-17 13-35-16.mp4' Pictures
'2023-10-15 16-44-39.mp4' '2023-10-17 13-41-12.mp4' Public
'2023-10-15 17-41-55.mp4' '2023-10-20 20-54-03.mp4' python
ooo@UBU-PC:~$
```

- I checked it through ls
- I want to go to desktop , so we do cd desktop
- ls -a in Linux**

The ls -a command is used to list **all files and directories**, including **hidden files**. Hidden files in Linux start with a dot (.), such as .bashrc or .config. By default, ls does not show these files, but ls -a displays everything in the current directory, including . (current directory) and .. (parent directory).

```
'2023-10-17 13-41-12.mp4'
ooo@UBU-PC:~$ ls -a
.          .bashrc      Music
..         .cache       .pcloud
'2023-10-08 15-50-29.mp4' .cert        Pictures
'2023-10-08 15-57-18.mp4' .config      .pki
'2023-10-08 16-03-47.mp4' Desktop      .profile
'2023-10-10 19-07-37.mp4' Documents    Public
'2023-10-12 19-35-14.mp4' Downloads    python
'2023-10-15 16-44-39.mp4' .fontconfig  .python_history
'2023-10-15 17-41-55.mp4' .gnome       .rediscli_history
'2023-10-16 18-13-35.mp4' .gnupg       .snap
'2023-10-17 13-32-42.mp4' .gphoto      snap
'2023-10-17 13-33-14.mp4' .ipynb_checkpoints .ssh
'2023-10-17 13-35-00.mp4' .ipython     .sudo_as_admin_successful
'2023-10-17 13-35-16.mp4' .java        Templates
'2023-10-17 13-41-12.mp4' .joplin      .var
'2023-10-20 20-54-03.mp4' .jupyter     Videos
                        .lessht          'VirtualBox VMs'
                        .local           .wget-hsts
                        .m2
                        .mongodb
                        .android
                        .audacity-data
                        .bash_history
                        .bash_logout
```

- In any folder , there is single dot and double dot
- Single dot (.) and double dot (..) in Linux**
- Single dot (.):** Represents the **current directory**. It is used to refer to files or commands in the directory you are currently working in.
- Double dot (..):** Represents the **parent directory** (one level up from the current directory). It is used to move or refer to files in the directory above.
- 👉 **Purpose:** They help in navigating directories and referencing file paths easily in Linux.
- To go to any folder we do cd Documents/

```
.bash_logout .mongodb
ooo@UBU-PC:~$ cd Documents/
ooo@UBU-PC:~/Documents$ pwd
/home/ooo/Documents
ooo@UBU-PC:~/Documents$
```

Master Linux Commands: A Beginner's Guide to ls, pwd, and

```
/home/ooo/Documents
ooo@UBU-PC:~/Documents$ ls
Books 'Obsidian Vault' Safe softwares 'Spring Boot' Test
ooo@UBU-PC:~/Documents$ ls -a
. .. Books 'Obsidian Vault' Safe softwares 'Spring Boot' Test
ooo@UBU-PC:~/Documents$
```



- Hence, if we do `cd ..` this means go one directory back

```
.bash_history .m2
.bash_logout .mongodb
ooo@UBU-PC:~$ cd Documents/
ooo@UBU-PC:~/Documents$ pwd
/home/ooo/Documents
ooo@UBU-PC:~/Documents$ ls
Books 'Obsidian Vault' Safe softwares 'Spring Boot' Test
ooo@UBU-PC:~/Documents$ ls -a
. .. Books 'Obsidian Vault' Safe softwares 'Spring Boot' Test
ooo@UBU-PC:~/Documents$ pwd
/home/ooo/Documents
ooo@UBU-PC:~/Documents$ cd ..
ooo@UBU-PC:~$ pwd
/home/ooo
ooo@UBU-PC:~$
```



```
ooo@UBU-PC:~/Documents$ ls -a
. .. Books 'Obsidian Vault' Safe softwares 'Spring B
ooo@UBU-PC:~/Documents$ ls -l
total 24
drwxrwxr-x 2 ooo ooo 4096 Aug 19 19:41 Books
drwxrwxr-x 3 ooo ooo 4096 Oct 7 22:35 'Obsidian Vault'
drwxrwxr-x 2 ooo ooo 4096 Sep 11 22:48 Safe
drwxrwxr-x 3 ooo ooo 4096 Oct 7 22:35 softwares
drwxrwxr-x 4 ooo ooo 4096 Oct 1 11:00 'Spring Boot'
drwxrwxr-x 2 ooo ooo 4096 Oct 18 20:06 Test
ooo@UBU-PC:~/Documents$
```

- Ls -l is for long format details


```
ooo@UBU-PC: ~/Documents
ooo@UBU-PC:~/Documents$ ls -al
total 32
drwxr-xr-x  8 ooo ooo 4096 Oct 20 20:58 .
drwxr-x--- 35 ooo ooo 4096 Oct 20 20:57 ..
drwxrwxr-x  2 ooo ooo 4096 Aug 19 19:41 Books
drwxrwxr-x  3 ooo ooo 4096 Oct  7 22:35 'Obsidian Vault'
drwxrwxr-x  2 ooo ooo 4096 Sep 11 22:48 Safe
drwxrwxr-x  3 ooo ooo 4096 Oct  7 22:35 softwares
drwxrwxr-x  4 ooo ooo 4096 Oct  1 11:00 'Spring Boot'
drwxrwxr-x  2 ooo ooo 4096 Oct 18 20:06 Test
ooo@UBU-PC:~/Documents$
```

-
- Ls -al shows both hidden and long format ones
- If we want to see human readable size , we do ls -alh

```
ooo@UBU-PC: ~/Documents
ooo@UBU-PC:~/Documents$ ls -alh
total 32K
drwxr-xr-x  8 ooo ooo 4.0K Oct 20 20:58 .
drwxr-x--- 35 ooo ooo 4.0K Oct 20 20:57 ..
drwxrwxr-x  2 ooo ooo 4.0K Aug 19 19:41 Books
drwxrwxr-x  3 ooo ooo 4.0K Oct  7 22:35 'Obsidian Vault'
drwxrwxr-x  2 ooo ooo 4.0K Sep 11 22:48 Safe
drwxrwxr-x  3 ooo ooo 4.0K Oct  7 22:35 softwares
drwxrwxr-x  4 ooo ooo 4.0K Oct  1 11:00 'Spring Boot'
drwxrwxr-x  2 ooo ooo 4.0K Oct 18 20:06 Test
ooo@UBU-PC:~/Documents$
```

-
- To sort the file view by time we can add t further
- Like ls -alht

```
ooo@UBU-PC: ~/Documents
ooo@UBU-PC:~/Documents$ ls -alht
total 32K
drwxr-x--- 35 ooo ooo 4.0K Oct 20 21:04 ..
drwxr-xr-x  8 ooo ooo 4.0K Oct 20 20:58 .
drwxrwxr-x  2 ooo ooo 4.0K Oct 18 20:06 Test
drwxrwxr-x  3 ooo ooo 4.0K Oct  7 22:35 'Obsidian Vault'
drwxrwxr-x  3 ooo ooo 4.0K Oct  7 22:35 softwares
drwxrwxr-x  4 ooo ooo 4.0K Oct  1 11:00 'Spring Boot'
drwxrwxr-x  2 ooo ooo 4.0K Sep 11 22:48 Safe
drwxrwxr-x  2 ooo ooo 4.0K Aug 19 19:41 Books
ooo@UBU-PC:~/Documents$
```

-
- Now sabse pehle newest arahe , sbse purana bad mai
- If I want to view whats inside some specific folder
- We write ls test/

```

ooo@UBU-PC:~/Documents$ man ls
ooo@UBU-PC:~/Documents$ ls
Books  'Obsidian Vault'  Safe  softwares  'Spring Boot'  Test
ooo@UBU-PC:~/Documents$ ls Test/
abc.txt
ooo@UBU-PC:~/Documents$

```

- I got no need of changing the directory , I viewed without changing directory to current folder

ROOT DIRECTORY & HOME DIRECTORY:

- IN OUR LINUX , IN OUR SYSTEM , THE FIRST PRENT FOLDER WHICH IS PRESENT IS CALLED ROOT DIRECTORY
- IT IS DENOTED BY /
- IF I DO CD /
- THIS MEANS I HAVE ARRIVED ATMOST PARENT

```

ooo@UBU-PC:~/Documents/Test$ cd ..
ooo@UBU-PC:~/Documents$ pwd
/home/ooo/Documents
ooo@UBU-PC:~/Documents$ cd /
ooo@UBU-PC:/$ ls
bin    dev    lib    libx32  mnt    root   snap   sys    var
boot   etc    lib32  lost+found  opt    run    srv    tmp
cdrom  home   lib64  media    proc   sbin   swapfile  usr
ooo@UBU-PC:/$ cd home/
ooo@UBU-PC:/home$ ls
ooo

```

- WHEN I OPEN THE LINUX TERMINAL , AND I WRITE PWD , THAT IS HOME DIRECTORY

```

ooo@UBU-PC:~$ pwd
/home/ooo
ooo@UBU-PC:~$

```

- / is root directory
- Like we can go to our root directory through cd /
- We can go to our home directory through cd ~

```
ooo@UBU-PC:~$ cd /
ooo@UBU-PC:/$ pwd
/
ooo@UBU-PC:/$ cd ~
ooo@UBU-PC:~$ pwd
/home/ooo
ooo@UBU-PC:~$
```

-
- With this we come to our home , user
- We see by ls , the folders in our directory , so we can on next command go to that folder through cd documents/
- However , other alternative of going to document folder is by absolute path

```
'2023-10-12 19-35-14.mp4' '2023-10-17 13-35-16.mp4' Pi
'2023-10-15 16-44-39.mp4' '2023-10-17 13-41-12.mp4' Pu
'2023-10-15 17-41-55.mp4' '2023-10-20 20-54-03.mp4' py
ooo@UBU-PC:~$ cd /home/ooo/Documents/
ooo@UBU-PC:~/Documents$
```

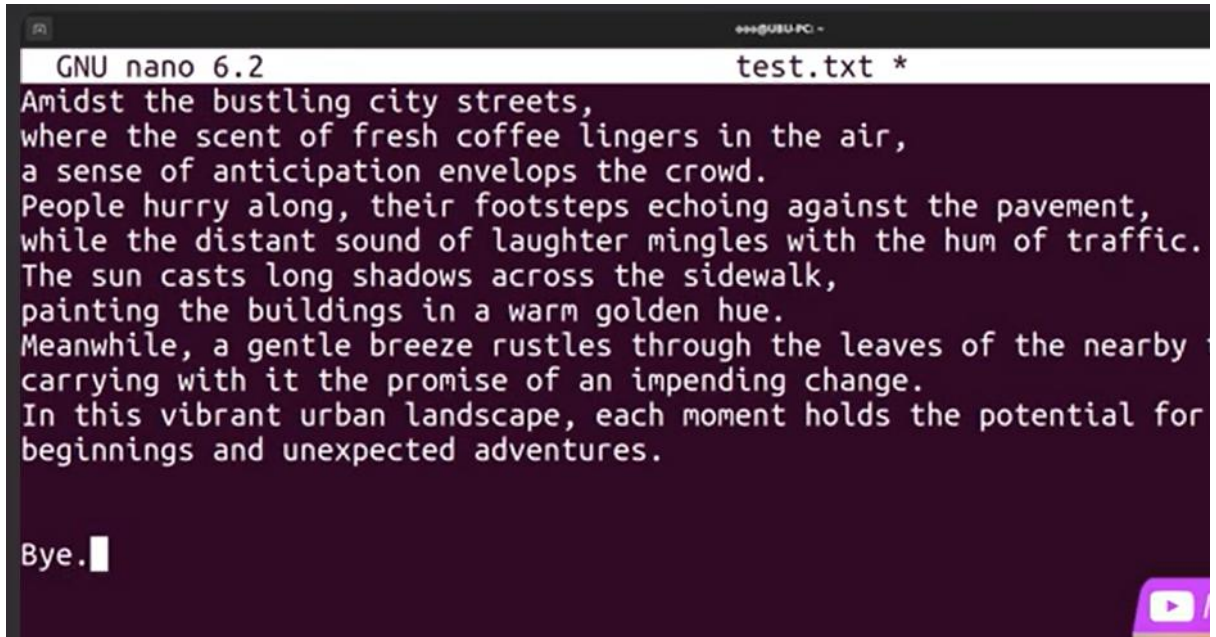
-
- The previous was relative path.

```
ooo@UBU-PC:~$ ls
'2023-10-08 15-50-29.mp4' '2023-10-16 18-13-35.mp4' Desktop snap
'2023-10-08 15-57-18.mp4' '2023-10-17 13-32-42.mp4' Documents Templates
'2023-10-08 16-03-47.mp4' '2023-10-17 13-33-14.mp4' Downloads Videos
'2023-10-10 19-07-37.mp4' '2023-10-17 13-35-00.mp4' Music 'VirtualBox VMs'
'2023-10-12 19-35-14.mp4' '2023-10-17 13-35-16.mp4' Pictures
'2023-10-15 16-44-39.mp4' '2023-10-17 13-41-12.mp4' Public
'2023-10-15 17-41-55.mp4' '2023-10-20 20-54-03.mp4' python
ooo@UBU-PC:~$ cd /home/ooo/Documents/
ooo@UBU-PC:~/Documents$ cd ..
ooo@UBU-PC:~$ pwd
/home/ooo
ooo@UBU-PC:~$ cd D
Desktop/ Documents/ Downloads/
ooo@UBU-PC:~$ cd Do
```

-

NANO TEXT EDITOR:

- While remaining on terminal , if you want to edit your text ffile , we can use nano text editor .
- If file is not created , it will be created
- We exit by control x



```
GNU nano 6.2 test.txt *
Amidst the bustling city streets,
where the scent of fresh coffee lingers in the air,
a sense of anticipation envelops the crowd.
People hurry along, their footsteps echoing against the pavement,
while the distant sound of laughter mingles with the hum of traffic.
The sun casts long shadows across the sidewalk,
painting the buildings in a warm golden hue.
Meanwhile, a gentle breeze rustles through the leaves of the nearby
carrying with it the promise of an impending change.
In this vibrant urban landscape, each moment holds the potential for
beginnings and unexpected adventures.

Bye.█
```

- If you write new thing in file and want to save it just press ctrl x a, then y

REMOVE , RENAME AND MOVE FILES AND FOLDERS:

Rm removes files and directories


```
ooo@UBU-PC:~$ ls
'2023-10-08 15-50-29.mp4' '2023-10-17 13-32-42.mp4'
'2023-10-08 15-57-18.mp4' '2023-10-17 13-33-14.mp4'
'2023-10-08 16-03-47.mp4' '2023-10-17 13-35-00.mp4'
'2023-10-10 19-07-37.mp4' '2023-10-17 13-35-16.mp4'
'2023-10-12 19-35-14.mp4' '2023-10-17 13-41-12.mp4'
'2023-10-15 16-44-39.mp4' '2023-10-21 17-10-22.mp4'
'2023-10-15 17-41-55.mp4' '2023-10-22 17-32-22.mp4'
'2023-10-16 18-13-35.mp4' '2023-10-22 17-33-07.mp4'
ooo@UBU-PC:~$ rm hello/
rm: cannot remove 'hello/': Is a directory
ooo@UBU-PC:~$ man rm
ooo@UBU-PC:~$ rm -d hello/
ooo@UBU-PC:~$
```

It deletes the directory

What if we have to create on file in hello folder

```
ooo@UBU-PC:~$ ls
'2023-10-08 15-50-29.mp4' '2023-10-17 13-32-42.mp4' a python
'2023-10-08 15-57-18.mp4' '2023-10-17 13-33-14.mp4' Desktop snap
'2023-10-08 16-03-47.mp4' '2023-10-17 13-35-00.mp4' Documents Templates
'2023-10-10 19-07-37.mp4' '2023-10-17 13-35-16.mp4' Downloads Videos
'2023-10-12 19-35-14.mp4' '2023-10-17 13-41-12.mp4' movies 'VirtualBox VMs'
'2023-10-15 16-44-39.mp4' '2023-10-21 17-10-22.mp4' Music
'2023-10-15 17-41-55.mp4' '2023-10-22 17-32-22.mp4' Pictures
'2023-10-16 18-13-35.mp4' '2023-10-22 17-33-07.mp4' Public
ooo@UBU-PC:~$ mkdir hello
ooo@UBU-PC:~$ touch hello/test.txt
ooo@UBU-PC:~$ ls
```

Now If we again do `rm -d hello/`

It will not delete hello , since hello directory is not empty and -d only deletes the empty directory

`Rm -r hello/` deletes the directory and its content recursively

Cp command copies files and directories

Sources aap kitne bhi desakte , destination aik hi denge

I want to copy up.mp4 and other.mp4 to action

```

ooo@UBU-PC:~/movies/kids/animation$ ls
up.mp4  zootopia.mp4
ooo@UBU-PC:~/movies/kids/animation$ cd ../../
ooo@UBU-PC:~/movies$ ls
action  comedy  horror  kids
ooo@UBU-PC:~/movies$ cd kids/
ooo@UBU-PC:~/movies/kids$ ls
animation
ooo@UBU-PC:~/movies/kids$ cd animation/
ooo@UBU-PC:~/movies/kids/animation$ ls
up.mp4  zootopia.mp4
ooo@UBU-PC:~/movies/kids/animation$ man cp
ooo@UBU-PC:~/movies/kids/animation$ cp up.mp4 zootopia.mp4 ../

```

Action is inside movies so I move one directory back and one more directory back , this means im at movies now

```

ooo@UBU-PC:~/movies/kids/animation$ ls
up.mp4  zootopia.mp4
ooo@UBU-PC:~/movies/kids/animation$ cd ../../
ooo@UBU-PC:~/movies$ ls
action  comedy  horror  kids
ooo@UBU-PC:~/movies$ cd kids/
ooo@UBU-PC:~/movies/kids$ ls
animation
ooo@UBU-PC:~/movies/kids$ cd animation/
ooo@UBU-PC:~/movies/kids/animation$ ls
up.mp4  zootopia.mp4
ooo@UBU-PC:~/movies/kids/animation$ man cp
ooo@UBU-PC:~/movies/kids/animation$ cp up.mp4 zootopia.mp4 ../../action/
ooo@UBU-PC:~/movies/kids/animation$ c

```

mv command is to move the files

MOVING & COPYING FOLDERS:

I Want to move kids folder to action folder ,

```

ooo@UBU-PC:~/movies$ ls
action  comedy  horror  kids
ooo@UBU-PC:~/movies$ cp kids/ action/
cp: -r not specified; omitting directory 'kids/'
ooo@UBU-PC:~/movies$ cp -r kids/ action/
ooo@UBU-PC:~/movies$ ls
action  comedy  horror  kids
ooo@UBU-PC:~/movies$ cd action/

```

Now , I want to move Zootopia to comedy directory , of movies folder

I go one step bak and then comdedy

✓ Correct command to move a folder:

`</> Bash`

```
mv folder1 folder2/
```

👉 This moves `folder1` inside `folder2`

```
ooo@UBU-PC:~/movies/action$ rm -rf kids/
ooo@UBU-PC:~/movies/action$ ls
up.mp4  zootopia.mp4
ooo@UBU-PC:~/movies/action$ cp zootopia.mp4 ../comedy/
```

Mv command can also be used to rename the folder

```
ooo@UBU-PC:~/movies/comedy$ ls
another.mp4  zootopia.mp4
ooo@UBU-PC:~/movies/comedy$ mv another.mp4 ../horror/another_2012.mp4
ooo@UBU-PC:~/movies/comedy$ ls
zootopia.mp4
ooo@UBU-PC:~/movies/comedy$ cd
```

Here I moved as normally , but with its addition , I wrote the rename folder

<code>`cp file.txt /tmp`</code>	Copy "file.txt" to the "/tmp" directory.
<code>`mv file.txt newname.txt`</code>	Rename "file.txt" to "newname.txt".
<code>`rm oldfile.txt`</code>	Remove the file "oldfile.txt".

Mv filename renamedifle

Same is with cp command

When you run:

```
bash
```

 Copy code

```
cp movie.mp4 mm.mp4
```

✓ If `mm.mp4` does NOT exist

- A new file named `mm.mp4` is created
- Contents of `movie.mp4` are copied into it
- Result: you now have **two files**

```
go
```

 Copy code

```
movie.mp4
```

```
mm.mp4 (copy of movie.mp4)
```

👉 This is **copying**, not renaming.

✓ If `mm.mp4` already exists

- `cp` will **overwrite** `mm.mp4`
- Old contents of `mm.mp4` are replaced
- `movie.mp4` remains unchanged



CAT COMMAND:

```
ooo@UBU-PC:~$ ls
'2023-10-23 19-21-49.mp4' Desktop Music snap
'2023-10-23 19-34-46.mp4' Documents Pictures Templates
'2023-10-23 19-35-09.mp4' Downloads Public Videos
'2023-10-23 19-51-00.mp4' movies python 'VirtualBox VMs'
ooo@UBU-PC:~$ nano a.txt
ooo@UBU-PC:~$ ls
'2023-10-23 19-21-49.mp4' a.txt movies python 'VirtualBox VMs'
'2023-10-23 19-34-46.mp4' Desktop Music snap
'2023-10-23 19-35-09.mp4' Documents Pictures Templates
'2023-10-23 19-51-00.mp4' Downloads Public Videos
ooo@UBU-PC:~$ nano b.txt
ooo@UBU-PC:~$
```



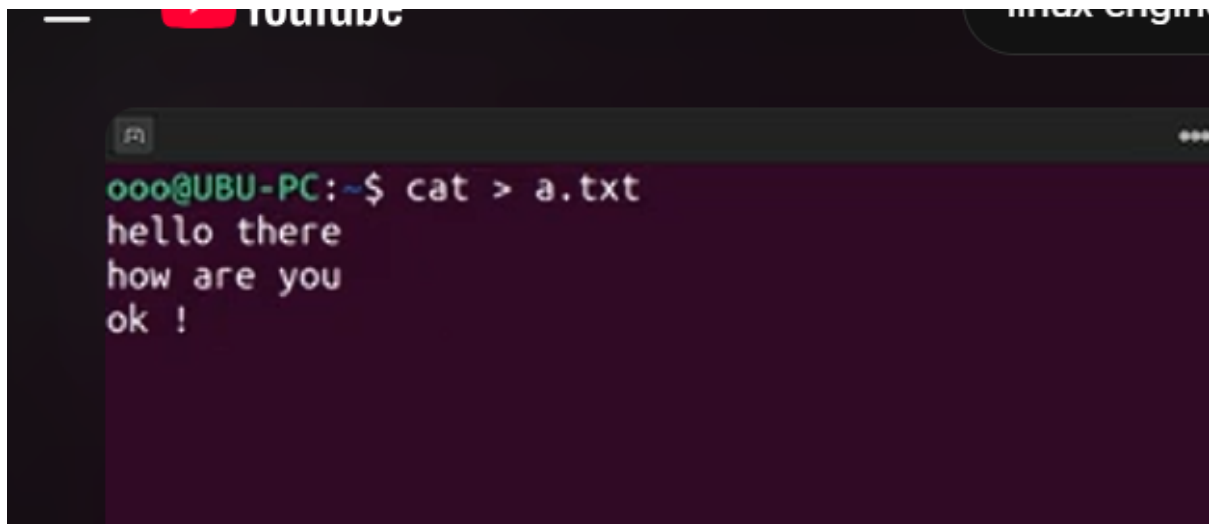
```
2023-10-23 19-34-46.mp4' a.txt
'2023-10-23 19-35-09.mp4' b.txt
ooo@UBU-PC:~$ cat a.txt b.txt
abc
xyz
ooo@UBU-PC:~$
```

If I do this , it will concatenate both the files

```
xyz
ooo@UBU-PC:~$ cat a.txt
abc
ooo@UBU-PC:~$
```

```
ooo@UBU-PC:~$ cat a.txt b.txt > c.txt
ooo@UBU-PC:~$ ls
'2023-10-23 19-21-49.mp4'  a.txt    Documents  Pictures
'2023-10-23 19-34-46.mp4'  b.txt    Downloads  Public
'2023-10-23 19-35-09.mp4'  c.txt    movies     python
'2023-10-23 19-51-00.mp4'  Desktop  Music      snap
ooo@UBU-PC:~$ cat c.txt
abc
xyz
ooo@UBU-PC:~$
```

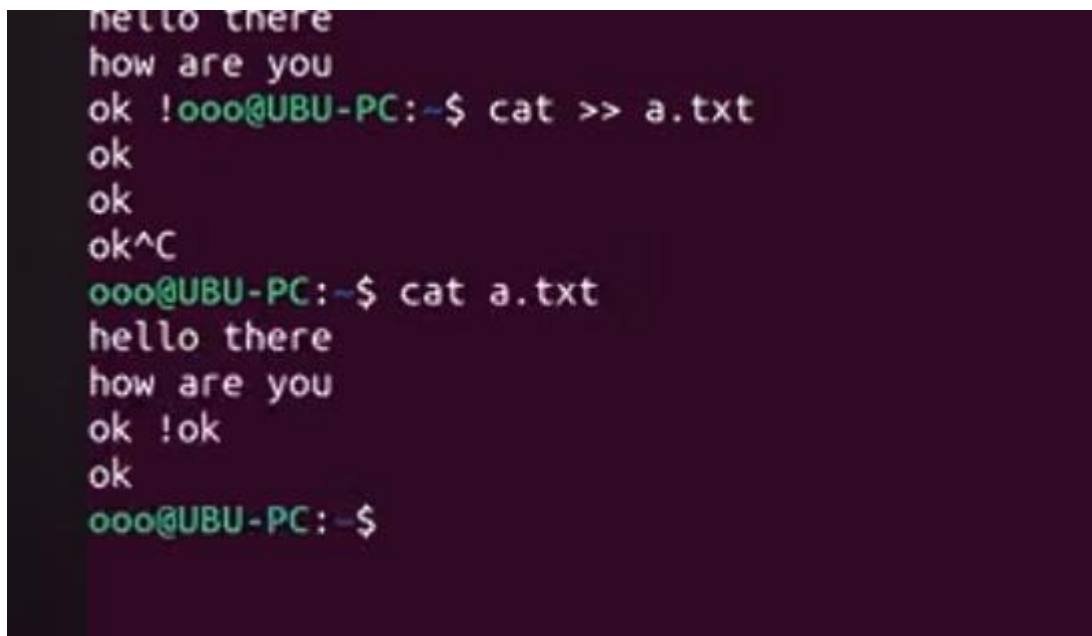
It means a.txt and b.txt ka concatenated text , c.txt mai daaldo c.txt ko create karke

A terminal window with a dark purple background. The prompt is 'ooo@UBU-PC:~\$'. The user enters 'cat > a.txt', followed by 'hello there', 'how are you', and 'ok !' on separate lines. The terminal shows the text being written to the file.

```
ooo@UBU-PC:~$ cat > a.txt
hello there
how are you
ok !
```

If I do `cat > a.txt`

It will directly help me to write the content in a.txt file . After im done , press `ctrl + d`

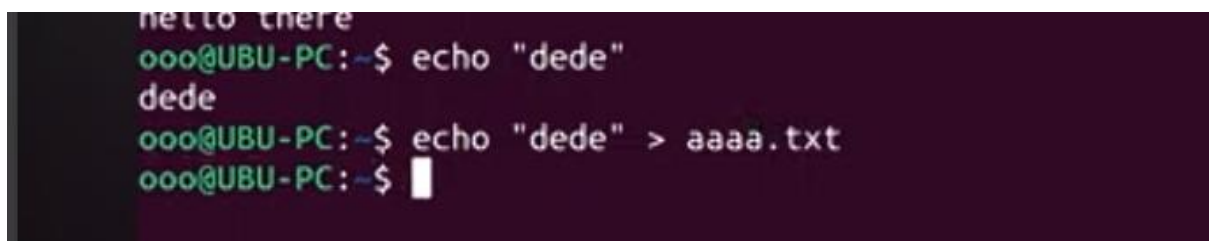
A terminal window showing the continuation of the previous session. The user enters 'ok !', then 'ooo@UBU-PC:~\$ cat >> a.txt', followed by 'ok', 'ok', and 'ok^C'. Then the user enters 'ooo@UBU-PC:~\$ cat a.txt', and the terminal displays the contents of the file: 'hello there', 'how are you', 'ok !', and 'ok'.

```
hello there
how are you
ok !ooo@UBU-PC:~$ cat >> a.txt
ok
ok
ok^C
ooo@UBU-PC:~$ cat a.txt
hello there
how are you
ok !
ok
ooo@UBU-PC:~$
```

If I do with double `>>` then it will not overwrite the content , but append them

Ctrl d means end of input

The same thing of creating the file newly and writing content to it can be done by echo

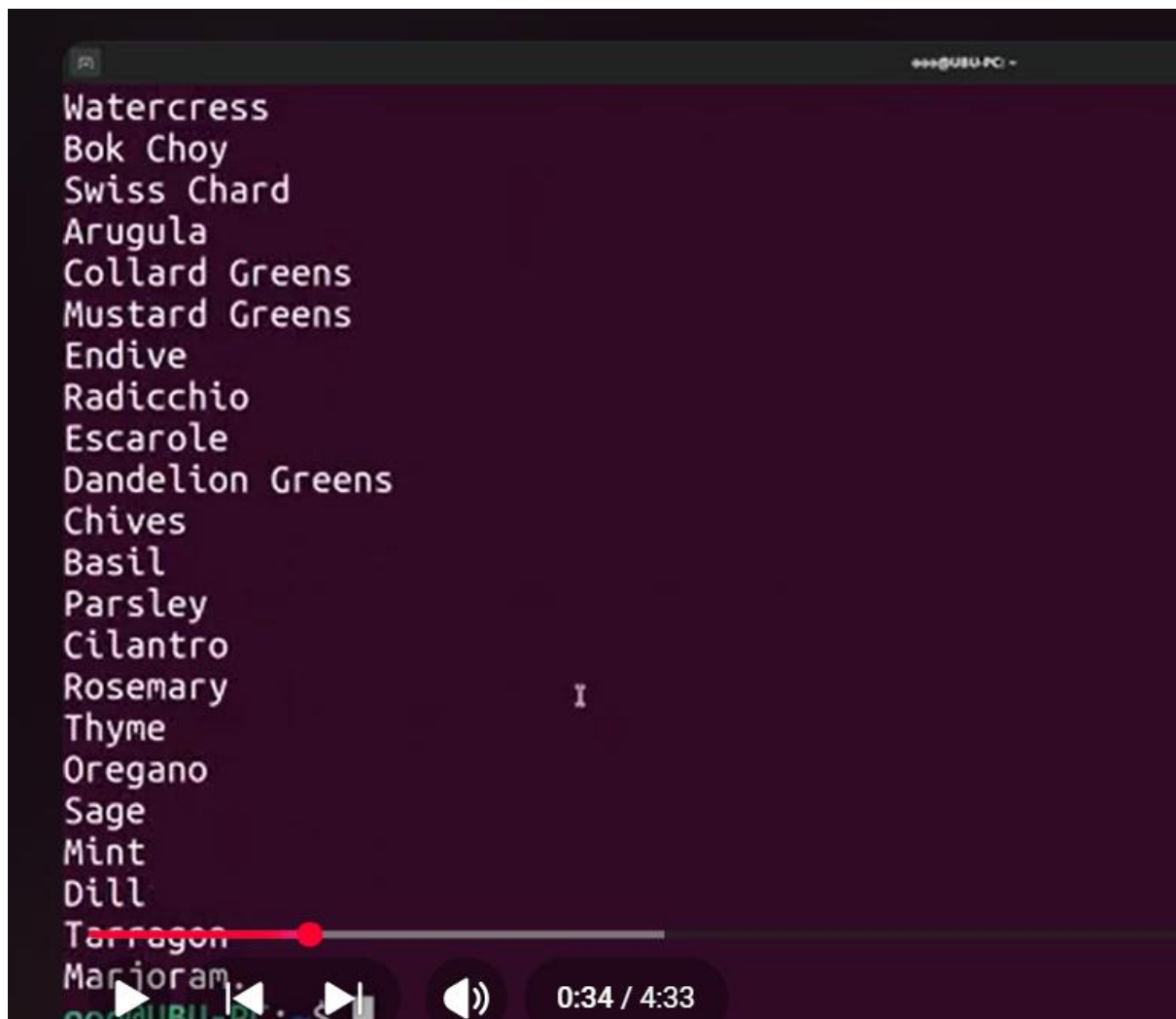
A terminal window showing the use of the 'echo' command. The user enters 'ooo@UBU-PC:~\$ echo "dede"', and the terminal outputs 'dede'. Then the user enters 'ooo@UBU-PC:~\$ echo "dede" > aaaa.txt', and the terminal shows the command being executed. The prompt 'ooo@UBU-PC:~\$' is shown again with a cursor.

```
hello there
ooo@UBU-PC:~$ echo "dede"
dede
ooo@UBU-PC:~$ echo "dede" > aaaa.txt
ooo@UBU-PC:~$
```

```
ooo@UBU-PC:~$ cat foods.txt
Pizza
Egg roll
Fries
Cake
Cookies
ooo@UBU-PC:~$ tac foods.txt
Cookies
Cake
Fries
Egg roll
Pizza
ooo@UBU-PC:~$ man
```

Print files in reverse

HEAD , TAIL COMMAND:



FOODS.TXT HAVE ALL THE THINGS , BUT I WANT TO SEE JUST THE FIRT 3 START THINGS OR 3 LAST THINGS


```
ooo@UBU-PC:~$ head foods.txt
Apple
Banana
Orange
Grapes
Watermelon
Pineapple
Mango
Strawberry
Blueberries
Raspberry
ooo@UBU-PC:~$ tail foods.txt
Parsley
Cilantro
Rosemary
Thyme
Oregano
Sage
Mint
Dill
Tarragon
Marjoram.
```

If I do head filename.txt

This means we get the first 10 values

If I want to see the specific line , we can use head -n 1 file.txt

```
ooo@UBU-PC:~$ head -n 1 foods.txt
Apple
ooo@UBU-PC:~$
```

```

ooo@UBU-PC:~$ head -n 1 foods.txt
Apple
ooo@UBU-PC:~$ head -1 foods.txt
Apple
ooo@UBU-PC:~$ tail -4 foods.txt
Mint
Dill
Tarragon
Marjoram.
ooo@UBU-PC:~$ █

```

Agar muje akhri ki 4 line dekhni to tail -4 file.txt

STANDARD OUTPUT:

When a program or command is executed in the terminal, it generates output that can be displayed directly in the terminal window. This output is known as the standard output.

```

ooo@UBU-PC:~$ ls
'2023-10-23 19-21-49.mp4'  '2023-10-26 22-28-30.mp4'  foods.txt  snap
'2023-10-23 19-34-46.mp4'  '2023-10-27 15-59-18.mp4'  movies     Templates
'2023-10-23 19-35-09.mp4'  '2023-10-27 19-48-25.mp4'  Music      Videos
'2023-10-24 18-28-38.mp4'  Desktop                     Pictures    'VirtualBox VMs'
'2023-10-24 18-31-12.mp4'  Documents                   Public
'2023-10-24 19-17-01.mp4'  Downloads                   python
ooo@UBU-PC:~$ pwd

```

THIS THING UNDER LS IS CALLED STANDARD OUTPUT

If I do ls > a.txt

All of ls things will get stored in a.txt

To do sorting of your file text , just write sort filename.txt

```
99
ooo@UBU-PC:~$ sort -n number.txt
2
3
09
9
10
33
99
100
200
```

LINUX PERMISSIONS:

 1. Viewing Permissions [Get Plus](#)

Use:

```
bash
ls -l
```

Example output:

```
bash
-rwxr-xr-- 1 user user 4096 Feb 12 file.sh
```

Breakdown:

```
powershell
- rwx r-x r--
| | | |
| | | |  Others (world)
| | | |  Group
| | | |  Owner
| | | |  File type
```

2. File Types (first character)

Symbol	Meaning
-	Regular file
d	Directory
l	Symbolic link

3. Permission Symbols

Symbol	Meaning	
r	Read	
w	Write	
x	Execute	

4. Permission Groups

Group	Who	
Owner (u)	File creator	
Group (g)	Users in same group	
Others (o)	Everyone else	

5. Changing Permissions (`chmod`)

Symbolic Method (easy to understand)

bash

 Copy code

```
chmod u+x file.sh    # Add execute to owner
chmod g-w file.sh    # Remove write from group
chmod o-r file.sh    # Set others to read only
chmod u+rw file.sh   # Give full permissions to owner
```

Symbols:

Operator	Meaning
+	Add permission
-	Remove permission
=	Set exact permission

Numeric (Octal) Method

Values:

Permission	Value
r	4
w	2
x	1

Example:

Add them:

Combo	Number
rwX	7
rw-	6
r-X	5
r--	4

Example:

```
bash
chmod 755 file.sh
```

Means:

```
css
Owner → 7 → rwX
Group → 5 → r-X
Others → 5 → r-X
```

What is `chown`?

[Get Plus](#) x

`chown` changes:

- ✓ Owner (user)
- ✓ Group

of a file or directory.

Basic Syntax

```
bash
chown [options] owner file
chown [options] owner:group file
```


✓ 1. Change Only Owner

bash

Copy code

```
sudo chown sadia file.txt
```

Meaning:

- ➡ File owner becomes **sadia**
- ➡ Group remains unchanged

⚠ Usually requires `sudo`

✓ 2. Change Owner AND Group

bash

Copy code

```
sudo chown sadia:developers file.txt
```

Meaning:

- ➡ Owner → sadia
- ➡ Group → developers

✓ 4. Apply Recursively (Very Important)

For directories:

bash

Copy code

```
sudo chown -R sadia:developers myfolder
```

➡ Changes ownership of:

- ✓ Folder
 - ✓ All files inside
 - ✓ All subdirectories
-

Check Ownership

```
bash
```

[Copy code](#)

```
ls -l
```

Example:

```
bash
```

[Copy code](#)

```
-rw-r--r-- 1 sadia developers 1200 Feb 12 file.txt
```

Columns:

```
pgsql
```

[Copy code](#)

```
owner → sadia
```

```
group → developers
```

What is `whoami`?

`whoami` shows:

✓ Current logged-in username

Basic Usage

```
bash
```

[Copy code](#)

```
whoami
```

✓ Example Output

```
bash
```

[Copy code](#)

```
sadia
```

Meaning:

 You are logged in as user sadia

Why is it Useful?

[Get Plus x](#)

✓ 1. Confirm Current User

Very helpful when:

- ✓ Using `sudo`
- ✓ Switching users
- ✓ Debugging permission issues

```
bash
```

[Copy code](#)

```
whoami
```

What is `su`?

[Get Plus](#) ×

`su` = Switch User

It allows you to:

- ✓ Change from current user → another user
- ✓ Often used to become **root**

Basic Syntax

```
bash
```

 Copy code

```
su [username]
```

If no username:

```
bash
```

 Copy code

```
su
```

 Defaults to **root**

✓ 1. Switch to Root

```
bash
```

[Copy code](#)

```
su
```

System asks:

```
makefile
```

[Copy code](#)

Password:

Enter root password

```
rohit@UBU-PC: ~$  
ooo@UBU-PC:/home$ whoami  
ooo  
ooo@UBU-PC:/home$ su - rohit  
Password:  
su: Authentication failure  
ooo@UBU-PC:/home$ su - rohit  
Password:  
rohit@UBU-PC:~$
```

```
su: Authentication failure  
ooo@UBU-PC:/home$ su - rohit  
Password:  
rohit@UBU-PC:~$ whoami  
rohit  
rohit@UBU-PC:~$ pwd  
/home/rohit  
rohit@UBU-PC:~$
```

LS – L shows all files plus permission

```
ooo@UBU-PC:~$ ls -ld /
-rw-rw-r-- 1 000 000 5696 Oct 29 20:15 meditations
drwxrwxr-x 6 000 000 4096 Oct 31 21:54 movies
drwxr-xr-x 3 000 000 4096 Oct 23 17:32 Music
-rw-rw-r-- 1 000 000 12346 Nov  2 17:04 mvn.srt
-rw-rw-r-- 1 000 000 0 Nov  2 20:05 one
drwxr-xr-x 5 000 000 4096 Nov 10 16:07 Pictures
drwxr-xr-x 2 000 000 4096 Jul 26 16:04 Public
drwxrwxr-x 3 000 000 4096 Jul 27 22:49 python
-rw-rw-r-- 1 000 000 3720 Oct 29 20:41 sample
drwx----- 19 000 000 4096 Oct  2 21:21 snap
drwxr-xr-x 2 000 000 4096 Jul 26 16:04 Templates
-rw-rw-r-- 1 000 000 0 Nov  2 20:05 thr
drwxr-xr-x 3 000 000 4096 Nov 10 18:57 Videos
drwxrwxr-x 5 000 000 4096 Sep 27 18:43 'VirtualBox VMs'
-rw-rw-r-- 1 000 000 0 Nov  2 20:05 wo
```

```
ooo@UBU-PC:~$ chown rohit sample
chown: changing ownership of 'sample': Operation not permitted
ooo@UBU-PC:~$ sudo chown rohit sample
[sudo] password for ooo:
ooo@UBU-PC:~$ chown rohit sample
```

We get root perms through sudo , without it don't work

◆ Real-life analogy 🏠

Think of:

- Directory = Room
- File = Book inside room

Even if:

- Book is readable 📖

If:

- Room door is locked 🔒 (no 🔑)

👉 You cannot reach the book

◆ Simple rule to remember

To access a file:

- "File needs permission ✅"
- "Directory ALSO needs 🔒 permission ✅"

◆ Quick demo idea

If directory has no execute:

```
<> Bash
chmod 744 Desktop
```

Now others:

- can see directory name ✗
- cannot enter ✗
- cannot access file ✗

✅ Final understanding

That line basically means:

👉 "Give others permission to ENTER the folder, otherwise they can't access the file even if file permissions allow it."

Users

fatima → Owner
ahmed → Other user

Step 1: File Creation (Owner)

```
cd Desktop
touch file1.txt
echo "hello, this is linux" > file1.txt
echo " how are you?" >> file1.txt
```

Step 2: Check Permissions

```
ls -l file1.txt
```

Output: -rw-rw-r-- 1 fatima fatima file1.txt

pwd -> check current working directory

ls -ld /home/fatima/Desktop/file1.txt --> check permission of directory where file exist.

If directory doesn't have execution permission for other-user then set permission for other users

```
chmod 755 /home/fatima/Desktop
```



```
chmod 755 /home/fatima/Desktop
```

Permission Visual

Owner	Group	Others
rw-	rw-	r--

Step 3: Create New User

```
sudo adduser ali
```

Step 4: Switch User

```
su - ali  
whoami
```

```
wnoami
```

Step 5: Other User Reads File

```
cat /home/fatima/Desktop/file1.txt
```

Visual Access Flow

```
fatima (owner)  
  |  
  v  
file1.txt  
  ^  
  |  
ali (read only)
```

FILE MANAGEMENT:

```
ooo@UBU-PC:~$ ls
'2023-10-08 15-50-29.mp4' '2023-10-16 18-13-35.mp4' '2023-10-21 18-05-03.mp4' python
'2023-10-08 15-57-18.mp4' '2023-10-17 13-32-42.mp4' Desktop snap
'2023-10-08 16-03-47.mp4' '2023-10-17 13-33-14.mp4' Documents Templates
'2023-10-10 19-07-37.mp4' '2023-10-17 13-35-00.mp4' Downloads Videos
'2023-10-12 19-35-14.mp4' '2023-10-17 13-35-16.mp4' Music 'VirtualBox VMs'
'2023-10-15 16-44-39.mp4' '2023-10-17 13-41-12.mp4' Pictures
'2023-10-15 17-41-55.mp4' '2023-10-21 17-10-22.mp4' Public

ooo@UBU-PC:~$ mkdir movies
ooo@UBU-PC:~$ ls
'2023-10-08 15-50-29.mp4' '2023-10-16 18-13-35.mp4' '2023-10-21 18-05-03.mp4' Public
'2023-10-08 15-57-18.mp4' '2023-10-17 13-32-42.mp4' Desktop python
'2023-10-08 16-03-47.mp4' '2023-10-17 13-33-14.mp4' Documents snap
'2023-10-10 19-07-37.mp4' '2023-10-17 13-35-00.mp4' Downloads Templates
'2023-10-12 19-35-14.mp4' '2023-10-17 13-35-16.mp4' movies Videos
'2023-10-15 16-44-39.mp4' '2023-10-17 13-41-12.mp4' Music 'VirtualBox VMs'
'2023-10-15 17-41-55.mp4' '2023-10-21 17-10-22.mp4' Pictures

ooo@UBU-PC:~$ cd movies/
```

```
ooo@UBU-PC:~/movies$ mkdir comedy action horror
ooo@UBU-PC:~/movies$ ls
action comedy horror
ooo@UBU-PC:~/movies$
```

THREE FOLDERS CREATED AT ONE TIME IN MOVIE DIRECTORY

I want to create kids folder in movies , and inside that kid folder , I need to make animation folder. And inside that animation folder , I want to create 2010

We are currently on movies folder

This will be done through `mkdir -p kids/animation/2010`

```
ooo@UBU-PC:~/movies$ ls
action comedy horror
ooo@UBU-PC:~/movies$ mkdir kids/animation
mkdir: cannot create directory 'kids/animation': No such file or directory
ooo@UBU-PC:~/movies$ man mkdir
ooo@UBU-PC:~/movies$ mkdir -p kids/animation/2010
ooo@UBU-PC:~/movies$ ls
action comedy horror kids
ooo@UBU-PC:~/movies$ cd kids/
ooo@UBU-PC:~/movies/kids$ ls
animation
ooo@UBU-PC:~/movies/kids$ cd animation/
ooo@UBU-PC:~/movies/kids/animation$ ls
2010
ooo@UBU-PC:~/movies/kids/animation$ cd 2010/
ooo@UBU-PC:~/movies/kids/animation/2010$ pwd
/home/ooo/movies/kids/animation/2010
ooo@UBU-PC:~/movies/kids/animation/2010$
```

We go back to home directory using `cd ~`

Touch command creates the file if its not created . but if existing file is touch , its time stamp is changed

```

agra.txt
ooo@UBU-PC:~/journal$ touch delhi.txt ny.txt dubai.txt
ooo@UBU-PC:~/journal$ ls
agra.txt delhi.txt dubai.txt ny.txt
ooo@UBU-PC:~/journal$ touch a.html
ooo@UBU-PC:~/journal$ ls
agra.txt a.html delhi.txt dubai.txt ny.txt
ooo@UBU-PC:~/journal$ touch hello
ooo@UBU-PC:~/journal$ ls
agra.txt a.html delhi.txt dubai.txt hello ny.txt
ooo@UBU-PC:~/journal$

```

We can create multiple files , also file with no extension appears as it uis

```

ooo@UBU-PC:~/journal$ ls
agra.txt a.html delhi.txt dubai.txt hello ny.txt
ooo@UBU-PC:~/journal$ stat agra.txt
  File: agra.txt
  Size: 0          Blocks: 0          IO Block: 4096   regular empty file
Device: 10302h/66306d Inode: 12076478 Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   ooo)   Gid: ( 1000/   ooo)
Access: 2023-10-21 18:14:08.946417547 +0530
Modify: 2023-10-21 18:14:08.946417547 +0530
Change: 2023-10-21 18:14:08.946417547 +0530
 Birth: 2023-10-21 18:13:12.494490250 +0530
ooo@UBU-PC:~/journal$

```

Stat command gives us access and modification time

Touch -m changes modification time while touch -a changes access time

Touch a b – would two files be created or 1

To create 1 , enclose it in quotients

```

'2023-10-15 17-41-55.mp4' '2023-10-21 17-10-22.mp4' Music 'Virt
ooo@UBU-PC:~$ touch 'a b'
ooo@UBU-PC:~$ ls
'2023-10-08 15-50-29.mp4' '2023-10-17 13-32-42.mp4' Desktop python
'2023-10-08 15-57-18.mp4' '2023-10-17 13-33-14.mp4' Documents snap
'2023-10-08 16-03-47.mp4' '2023-10-17 13-35-00.mp4' Downloads Templates
'2023-10-10 19-07-37.mp4' '2023-10-17 13-35-16.mp4' journal Videos
'2023-10-12 19-35-14.mp4' '2023-10-17 13-41-12.mp4' movies 'VirtualBox VMs'
'2023-10-15 16-44-39.mp4' '2023-10-21 17-10-22.mp4' Music
'2023-10-15 17-41-55.mp4' '2023-10-21 18-05-03.mp4' Pictures
'2023-10-16 18-13-35.mp4' 'a b' Public
ooo@UBU-PC:~$

```

File command gives us the file type

```

'2023-10-15 16-44-39.mp4' '2023-10-21 17-10-22.mp4' Music
'2023-10-15 17-41-55.mp4' '2023-10-21 18-05-03.mp4' Pictures
'2023-10-16 18-13-35.mp4' 'a b' Public
ooo@UBU-PC:~$ file '2023-10-21 18-05-03.mp4'
2023-10-21 18-05-03.mp4: ISO Media, MP4 Base Media v1 [ISO 14496-12:2003]
ooo@UBU-PC:~$

```

```
'2023-10-15 17-41-55.mp4' '2023-10-21 18-05-03.mp4' Mus
'2023-10-16 18-13-35.mp4' 'a b' Pic
ooo@UBU-PC:~$ cat abc
hello
ooo@UBU-PC:~$ file abc
abc: ASCII text
```

Abc.mp4 is created , but is left empty .

```
abc: ASCII text
ooo@UBU-PC:~$ touch abc.mp4
ooo@UBU-PC:~$ file abc.mp4
abc.mp4: empty
```

 **3** **Compile a C Program (gcc)**

 **Step 1: Write code**

```
bash
```

```
nano hello.c
```

Copy code

Example:

```
c
```

```
#include <stdio.h>

int main() {
    printf("Hello from C!\n");
    return 0;
}
```

Copy code

Save → **CTRL + X** → **Y** → Enter

Default output:

```
csharp
a.out
```

Copy code

 **Step 3: Run**

```
bash
./a.out
```

Copy code

Output:

```
csharp
Hello from C!
```

Copy code

SECOND WAY TO COMPILE C FILE

  **Compile C Program with Custom Name**

```
bash
gcc hello.c -o hello
```

Copy code

Run:

```
bash
./hello
```

Copy code

C++ file execution



Compile a C++ Program (g++)

Get Plus x



Step 1: Write code

bash

Copy code

nano hello.cpp

Example:

cpp

Copy code

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello from C++!" << endl;
    return 0;
}
```



Step 2: Compile

bash

Copy code

g++ hello.cpp -o hello



Step 3: Run

bash

Copy code

./hello



What is a Makefile?

A **Makefile** is a file used by the `make` utility to:

- ✓ Automate compilation
- ✓ Save typing long commands
- ✓ Rebuild only changed files

Think of it as:



"A recipe book for building your program"

😬 Why Do We Need It?

Without Makefile:

```
bash
```

[Copy code](#)

```
g++ main.cpp functions.cpp utils.cpp -o program
```

Every time. 😬

With Makefile:

```
bash
```

[Copy code](#)

```
make
```

Done. 😎



3

Compile a C Program (gcc)



Step 1: Write code

```
bash
```

Copy code

```
nano hello.c
```

Example:

```
c
```

Copy code

```
#include <stdio.h>

int main() {
    printf("Hello from C!\n");
    return 0;
}
```

Save → **CTRL + X** → **Y** → Enter

MAKEFILE:

Makefile Example with C Program

Project Structure:

```
project/  
├── main.c  
├── sum.c  
├── sum.h  
└── Makefile
```

sum.h (Function declaration)

```
int sum(int a, int b);
```

sum.c (Function definition)

```
#include "sum.h"
```

```
int sum(int a, int b)  
{  
    return a + b;  
}
```

sum.h (Function declaration)

```
int sum(int a, int b);
```

sum.c (Function definition)

```
#include "sum.h"
```

```
int sum(int a, int b)
{
    return a + b;
}
```

main.c (Main program file)

```
#include <stdio.h>
```

```
#include "sum.h"
```

```
int main()
{
    int result = sum(2, 3);
    printf("Sum = %d\n", result);
    return 0;
}
```

Makefile (Makefile to compile the program)

program: main.o sum.o

gcc main.o sum.o -o program

main.o: main.c sum.h

gcc -c main.c

sum.o: sum.c sum.h

gcc -c sum.c

clean:

sum.o: sum.c sum.h

gcc -c sum.c

clean:

Machine Code (.o, .obj) ↓

rm -f *.o program

```
rm -f *.o program
```

How to Run

1. Open terminal and cd to project directory (create it if not exist)
2. Run: `make` ---> for compilation and execution of commands
3. Run program: `make run` --> for the output
4. Clean files: `make clean` ---> to remove all object and output files

1. What is a Makefile?

A **Makefile** is a special file used by the `make` utility to automate the compilation and build process in programming projects.

Why it's useful:

1. **Automation** – You don't have to type long `gcc` commands every time.
2. **Efficiency** – Only recompiles files that have changed instead of recompiling everything.
3. **Dependency management** – Tracks which files depend on which other files, e.g., if you change a header `.h` file, only the affected `.c` files are recompiled.
4. **Portability** – You can move your project to another system, and `make` will know how to build it.

2. Basic Compilation Without Makefile

Suppose we have these files:

- `hellomake.c` → main program
- `hellofunc.c` → additional functions
- `hellomake.h` → header file

To compile them manually:

```
bash
```

[Copy code](#)

```
gcc -o hellomake hellomake.c hellofunc.c -I
```

- `gcc` → GNU C compiler
- `-o hellomake` → name of the output executable
- `hellomake.c hellofunc.c` → source files to compile
- `-I` → tells the compiler to look for header files in a directory (like current directory `.`).

Problem:

- If you modify `hellofunc.c`, you still compile everything again – **inefficient**.
- If you modify `hellomake.h`, nothing happens unless you recompile all manually.

1. Why recompiling everything is inefficient

Imagine you have a program with 3 files:

- `hellomake.c` → main program
- `hellofunc.c` → functions
- `hellomake.h` → header file

Case 1: You modify `hellofunc.c`

If you compile manually like this every time:

```
bash
gcc -o hellomake hellomake.c hellofunc.c -I.
```

Copy code

- This command **recompiles** `hellomake.c` even though you didn't change it.
- `hellomake.c` might be large, so recompiling it takes time.
- Only `hellofunc.c` actually needed compilation, but the command doesn't know that.

Inefficient → wasting time compiling files unnecessarily.

Case 2: You modify `hellomake.h`

Suppose `hellomake.c` and `hellofunc.c` include `hellomake.h` like this:

```
c
#include "hellomake.h"
```

Copy code

If you **only modify** `hellomake.h` and run:

```
bash
gcc -o hellomake hellomake.c hellofunc.c -I.
```

Copy code

- Manually, you **have to remember to recompile both** `.c` files that include the header.
- If you forget one, your program might behave incorrectly, or the change won't take effect.

Problem → manually tracking dependencies is error-prone.

2. How to open/create a Makefile

A **Makefile** is just a text file named `Makefile` (capital M) or `makefile` (lowercase m).

You can create or edit it using any text editor. Here's how:

Using Nano (terminal editor)

1. Open terminal in your project folder:

```
bash
```

[Copy code](#)

```
cd /path/to/your/project
```

2. Create/open a Makefile:

```
bash
```

[Copy code](#)

```
nano Makefile
```

3. This opens the editor. You can type your Makefile content here.

4. Save and exit:

- Press `Ctrl + O` → hit `Enter` to save
- Press `Ctrl + X` → exit Nano

3. Makefile 1 – First Simple Version

```
makefile
```

[Copy code](#)

```
hellomake: hellomake.c hellofunc.c
    gcc -o hellomake hellomake.c hellofunc.c -I
```

Explanation:

- `hellomake:` → Target name (the executable you want to build).
- `hellomake.c hellofunc.c` → Dependencies (if any of these files change, rebuild target).
- The next line (with a **tab** at the start!) is the command to run to build the target.

✅ Benefit: now you can just type `make` instead of writing the full `gcc` command.

4. Makefile 2 – Using Variables and Object Files

makefile

Copy code

```
CC = gcc
CFLAGS = -I
hellomake: hellomake.o hellofunc.o
    $(CC) -o hellomake hellomake.o hellofunc.o
```

New Concepts:

1. Variables (`CC` and `CFLAGS`)

- `CC = gcc` → sets compiler to GCC
- `CFLAGS = -I` → additional compilation flags
- Using variables makes it easy to change compiler/flags later

2. Object files (`.o`)

- `hellomake.o` and `hellofunc.o` are object files generated from `.c` files.
- Object files allow **partial compilation**, only recompiling the file that changed.

3. Recipe using variables

makefile

Copy code

```
$(CC) -o hellomake hellomake.o hellofunc.o
```

is equivalent to

bash

Copy code

```
gcc -o hellomake hellomake.o hellofunc.o
```

5. Makefile 3 – Adding Dependencies on Header Files

makefile

Copy code

```
CC = gcc
CFLAGS = -I.
DEPS = hellomake.h

%.o: %.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)

hellomake: hellomake.o hellofunc.o
    $(CC) -o hellomake hellomake.o hellofunc.o
```

New Concepts:

1. Header Dependencies (`DEPS`)

- `.h` files can affect compilation. If `hellomake.h` changes, you need to recompile dependent `.c` files.
- `DEPS = hellomake.h` defines that dependency.

2. Pattern Rule (`%.o: %.c $(DEPS)`)

- `%` is a wildcard
- This says: "To build any `.o` file, look at the corresponding `.c` file and `DEPS` files."
- Example: `hellomake.o` depends on `hellomake.c` and `hellomake.h`.

- This says: “To build any `.o` file, look at the corresponding `.c` file and `DEPS` files.”
- Example: `hellomake.o` depends on `hellomake.c` and `hellomake.h`.

3. Automatic variables

makefile

 Copy code

```
$@ → target filename (e.g., hellomake.o)
$< → first prerequisite (e.g., hellomake.c)
```

Recipe explanation:

makefile

 Copy code

```
$(CC) -c -o $@ $< $(CFLAGS)
```

- `-c` → compile but don't link
- `-o $@` → output file = target
- `$<` → input file = first dependency
- `$(CFLAGS)` → compiler flags

✓ Benefit: if you add more `.c` files, you don't need to write a separate compile command for each.

6. Makefile 4 – Using `$@` and `$^` for Full Automation

makefile

 Copy code

```
CC = gcc
CFLAGS = -I.
DEPS = hellomake.h
OBJ = hellomake.o hellofunc.o

%.o: %.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)

hellomake: $(OBJ)
    $(CC) -o $@ $^ $(CFLAGS)
```

New Concepts:

1. `$^` → all prerequisites
 - If your target depends on multiple files (`$(OBJ)`), `$^` expands to all of them.
 - Example: `$^ = hellomake.o hellofunc.o`
2. `OBJ` variable
 - Lists all object files needed to build the executable
3. Fully automated workflow
 - Changing `hellomake.c` → only `hellomake.o` is rebuilt, then linked into executable
 - Changing `hellomake.h` → both `hellomake.o` and `hellofunc.o` are rebuilt

4. General benefits

4. General benefits

Feature	Benefit
Variables (<code>CC</code> , <code>CFLAGS</code>)	Easy to change compiler or flags
Object files (<code>.o</code>)	Only rebuild what's needed
Header dependencies (<code>DEPS</code>)	Recompile when <code>.h</code> files change
Pattern rules (<code>%.o: %.c</code>)	Automates compilation for all files
Automatic variables (<code>\$@</code> , <code>^</code>)	Makes the makefile cleaner and scalable

Project Structure

```
css
Copy code

project/
├─ main.c   → main program
├─ sum.c    → function implementation
├─ sum.h    → function declaration (header)
└─ Makefile → commands to build the program
```

Files:

- `sum.h` → declares function `int sum(int a, int b);`
- `sum.c` → defines the function `sum`
- `main.c` → calls `sum` and prints result
- `Makefile` → tells `make` how to compile and link the program

The Makefile

```
makefile
Copy code

program: main.o sum.o
    gcc main.o sum.o -o program
```

Breakdown:

1. **Target:** `program` → the executable you want to create.
2. **Dependencies:** `main.o sum.o` → if any of these `.o` files are missing or updated, rebuild `program`.
3. **Recipe:** `gcc main.o sum.o -o program`
 - Links the object files into an executable called `program`.
 - This command **only runs if the target is out of date**, i.e., `main.o` or `sum.o` changed.

✅ Benefit: You don't rebuild the executable unless necessary.

makefile

 Copy code

```
main.o: main.c sum.h
gcc -c main.c
```

Breakdown:

1. Target: `main.o` → the object file from `main.c`.
2. Dependencies: `main.c sum.h`
 - If `main.c` or `sum.h` changes, `main.o` needs to be recompiled.
 - This is important because `main.c` includes `sum.h`, so changes in the header affect compilation.
3. Recipe: `gcc -c main.c`
 - `-c` tells gcc: **compile only** (don't link) → generates `main.o`.

✓ Benefit: Only compiles what's necessary.

makefile

 Copy code

```
sum.o: sum.c sum.h
gcc -c sum.c
```

Breakdown:

1. Target: `sum.o` → object file from `sum.c`.
2. Dependencies: `sum.c sum.h`
 - Recompile `sum.o` if either `sum.c` or `sum.h` changes.
3. Recipe: `gcc -c sum.c`
 - Generates `sum.o` without linking.

✓ Benefit: Efficient compilation — only rebuild changed code.

So target are program , main.o and sum.o

- Generates `sum.o` without linking.

✓ Benefit: Efficient compilation — only rebuild changed code.

makefile

 Copy code

```
clean:
rm -f *.o program
```

Breakdown:

1. Target: `clean` → a "phony" target (not a file, just a command).
2. Recipe: `rm -f *.o program`
 - Deletes all object files (`*.o`) and the executable (`program`).
3. Usage:

bash

 Copy code

```
make clean
```

- Useful to start from scratch if things are messed up.

How It Works Together

1. You type:

```
bash
```

 Copy code

```
make
```

2. `make` sees the first target (`program`) → needs `main.o` and `sum.o`

3. Checks `main.o`:

- If missing or `main.c` / `sum.h` changed → runs `gcc -c main.c`

4. Checks `sum.o`:

- If missing or `sum.c` / `sum.h` changed → runs `gcc -c sum.c`

5. After both `.o` files are up-to-date → runs `gcc main.o sum.o -o program` to link executable.

Step-by-Step Usage

[+ Get Plus](#) [×](#)

1. Open terminal, navigate to project folder:

```
bash
```

 Copy code

```
cd /path/to/project
```

2. Compile program:

```
bash
```

 Copy code

```
make
```

3. Run program:

```
bash
```

 Copy code

```
./program
```

or if you added `run` target:

```
bash
```

 Copy code

```
make run
```

4. Clean project:

```
bash
```

 Copy code

```
make clean
```

- Deletes `.o` files and `program` executable



SHELL SCRIPTING

What even IS Shell Scripting?

Think of your computer like a very obedient servant. Normally you give it **one command at a time** by typing in the terminal:

```
bash

ls      # show files
pwd     # where am I?
mkdir foo # make a folder
```

Shell Scripting = writing all those commands in a **file** so they run automatically, one after another — like a recipe 🍳

That file is called a **shell script** (ends in `.sh`)

◆ 1. BASIC SYSTEM VARIABLES

 Bash

```
echo "User Name : $USER"
echo "Home Directory : $HOME"
echo "Current Directory: $PWD"
echo "Shell Being Used : $SHELL"
echo "Path Variable : $PATH"
```

✅ Purpose:

Shows information about your system/user environment

Meaning:

- `$USER` → current logged-in user
- `$HOME` → your home folder
- `$PWD` → current working directory
- `$SHELL` → which shell you are using (e.g., bash)
- `$PATH` → list of directories where commands are searched

Example output:

```
User Name : mujtaba
Home Directory : /home/mujtaba
Current Directory: /home/mujtaba/Desktop
Shell Being Used : /bin/bash
Path Variable : /usr/bin:/bin:/usr/local/bin
```

The absolute basics — before any code

Concept	Plain English
<code>#!/bin/bash</code>	"Hey computer, use bash to run this file" — always line 1
<code>#</code>	A comment — ignored by computer, just for humans
<code>echo</code>	Print something to screen
<code>read</code>	Take input from user
Variables	Boxes that store values
<code>if/else</code>	Make decisions
Loops	Repeat things

◆ 2. SCRIPT & ARGUMENT VARIABLES

 Bash

```
echo "Script Name : $0"
echo "Total Arguments : $#"
```

```
echo "All Arguments : $@"
```

✓ Purpose:

Handles input given to the script

🔍 Meaning:

- `$0` → script name
- `$#` → number of arguments
- `$@` → all arguments

Example run:

 Bash

```
./script.sh hello world
```

Output:

```
Script Name : ./script.sh
Total Arguments : 2
All Arguments : hello world
```

Example run:

```
</> Bash
./script.sh hello world
```



Output:

```
Script Name : ./script.sh
Total Arguments : 2
All Arguments : hello world
```



◆ Process ID \$\$

```
</> Bash
echo "Current Shell PID : $$"
```



👉 Every running program has an ID
This gives **your script's process ID**

Example:

```
Current Shell PID : 3452
```



◆ Last background process `$!`

```
</> Bash
sleep 5 &
echo "Last Background Process PID : $!"
```

What happens:

- `sleep 5 &` → runs in background
- `$!` → gives its PID

Example:

```
Last Background Process PID : 3453
```

◆ 1 Command Status → `$?`

Purpose:

`$?` tells you whether the last command executed successfully or failed.

How it works:

- After you run any command, `$?` automatically stores its **exit status**.
- Convention:
 - `0` → command succeeded ✓
 - Non-zero → command failed ✗

Example 1: Success

```
</> Bash
ls /home/mujtaba
echo "Exit status: $?"
```

- `ls` shows the folder contents ✓
- `$?` → `0`

Output:

```
Exit status: 0
```

Example 2: Error

 Bash

```
ls /fakefolder  
echo "Exit status: $?"
```

- Folder does not exist ❌
- `$?` → `2` (or some non-zero number)

Output:

```
ls: cannot access '/fakefolder': No such file or directory  
Exit status: 2
```



Key idea:

`$?` always remembers the status of the last command only, so if you run another command before checking, it will change.

◆ 2 Script running time → `$SECONDS`

Purpose:

`$SECONDS` gives how many seconds have passed since the script started running.

How it works:

- Bash automatically creates `$SECONDS` for every script
- Starts at `0` when script begins
- Increases **1 every second**

Example:

 Bash

```
#!/bin/bash  
  
echo "Script start: $SECONDS seconds"  
sleep 3  
echo "After 3 seconds: $SECONDS seconds"  
sleep 2  
echo "After total 5 seconds: $SECONDS seconds"
```

Run output:

```
Script start: 0 seconds  
After 3 seconds: 3 seconds  
After total 5 seconds: 5 seconds
```



⚡ Notes:

- You don't need to calculate anything; Bash does it automatically
- If you want to reset `$SECONDS` :

↔ Bash

`SECONDS=0`



- Now counting starts from 0 again

```
#!/bin/bash
```

```
read -p "Enter a number (1-3): " num
```

```
case $num in
```

```
1)
```

```
    echo "You chose One"
```

```
    ;;
```

```
2)
```

```
    echo "You chose Two"
```

```
    ;;
```

```
3)
```

```
    echo "You chose Three"
```

```
    ;;
```

```
*)
```

```
    echo "Invalid choice"
```

```
    ;;
```

```
esac
```

BASICS

CONSTANT VARIABLE?

Once you defined a variable and don't wanna change it until end of the script.

```
readonly var_name="Hi"
```

◆ Taking input with prompt (shortcut)

</> Bash

```
read -p "Enter your name: " name  
echo "Hello $name"
```

👉 -p shows the message on the same line

IF-ELSE

```
if [ $age -eq 18 ]  
then  
    echo "You are eligible to vote"  
else  
    echo "Sorry you are not eligible to vote"  
fi
```

ELIF

```
if [ $country = "India" ]  
then  
    echo "You are Indian"  
elif [ $country = "Nepal" ]  
then  
    echo "You are from Nepal" ➡  
else  
    echo "You are from Earth"  
fi
```

CHECK IF FILE/DIR EXIST

```
if [ -d folder_name] If folder exists  
[ ! -d folder_name] If folder not exists  
  
if [ -f file_name] If file exists  
if [ ! -f file_name] If file not exists
```

MPRASHANT

SWITCH

```
echo "Hey choose an option"  
echo "a = To see the current date"  
echo "b = list all the files in current dir"  
  
read choice  
  
case $choice in  
    a) date;;  
    b) ls;;  
    *) echo "Non a valid input"  
esac
```

MPRASHANT

FOR LOOP

```
for i in 1 2 3 4 5  
do  
    echo "Number is $i"  
done
```

Other ways to write For loop

```
for j in Raju Sham Baburao  
for p in {1..20}
```


WHILE LOOP

```
count=0
num=10

while [ $count -le $num ]
do
    echo "Numbers are $count"
    let count++
done
```



INFINITE LOOP..

```
while true
do
    echo "Hi"
    sleep 2s
done
```



HOW TO USE EXPRESSIONS

Using let command

Example:

```
let a++  
let a=5*10
```

PRINT NAME OF THE SCRIPT

```
echo "The name of the script is: ${0}"
```

HOW TO MAKE FUNCTIONS?

```
function myfun {  
    echo "Hi"  
}
```

```
myfun() {  
    echo "Hello"  
}
```

To call the function
myfun



HOW TO USE ARGUMENTS IN FUNCTIONS?

```
addition() {  
    local num1=$1  
    local num2=$2  
    let sum=$num1+$num2  
    echo "Sum of $num1 and $num2 is $sum"  
}
```

```
myfun 12 13
```



ARGUMENTS IN SCRIPT...

```
#myscript.sh arg1 arg2..
```

How to access these arguments inside
our script?

To get no. of arguments : \$#

To display all arguments : \$@

To use or display a argument: \$1 \$2 ..



BASH VARIABLES...

RANDOM - A random integer between 0
and 32767 is generated



UID - User ID of the user logged in



```
[paul@cs8 cc]$  
[paul@cs8 cc]$ echo $RANDOM  
22305  
[paul@cs8 cc]$ echo $RANDOM  
18619  
[paul@cs8 cc]$ █
```

```
~~~~~  
[paul@cs8 cc]$ echo $UID  
1000  
[paul@cs8 cc]$ █
```

RASHANT

In case if you don't wanna print the output of a command on terminal or write in a file,

WHAT IS
/DEV/NULL

We can redirect the output to
`/dev/null`

Example:

`#cd /root &> /dev/null`



USEFUL CONCEPTS...



sleep - to create delay between two executions ex: sleep 1s/1m

exit - to stop script at a point



exit status **\$?** - gives you status of previous command if that was successful

◆ Meaning of values

Value	Meaning
0	✓ Success
non-zero (1,2,...)	✗ Error / Failure

◆ Example 1 (Success)

❏ Bash

```
ls  
echo $?
```

Output:

0

👉 ls worked → success → 0

◆ Example 2 (Failure)



❏ Bash

```
ls_wrongfolder
```

LOG MESSAGES..



If you want to maintain the logging for your script, you can use **logger** in your script.

You can find the logs under **/var/logs/messages**

Example: **#logger "Hey Buddy"**



When I write hey buddy it will go to messages

1. Hello World & Variables

Code:

```
#!/bin/bash
name="Fatima"
echo "Hello $name"
```

Execution:

```
chmod +x hello.sh
./hello.sh
```

Output:

```
Hello Fatima
```

SCRIPT 1: Hello World & Variables

Concept First: What is a Variable?

A **variable** is just a **named box** that stores a value.

```
name = "Fatima"
```

Imagine a box labeled `name`, and inside it is the word `Fatima`.

When you write `$name`, you're saying "give me what's inside that box".

Code Breakdown — Line by Line

```
bash
```

```
#!/bin/bash
```

✳ Always line 1. Tells the computer: "Use bash to run this script." Called a shebang. Never forget it!

```
bash
```

```
name="Fatima"
```

📦 Creates a variable called `name` and stores `"Fatima"` in it. ⚠ NO spaces around `=`. `name = "Fatima"` will break!

```
bash
```

```
echo "Hello $name"
```

🖨 `echo` = print to screen. `$name` gets replaced with `Fatima`. Output → `Hello Fatima`

◆ Correct steps (in order)

1. Create the file

```
</> Bash
touch hello.sh
```

2. Add code inside it

(using nano or any editor)

```
</> Bash
nano hello.sh
```

Paste:

```
</> Bash
#!/bin/bash
name="Fatima"
echo "Hello $name"
```

Save and exit.

3. Make it executable

```
</> Bash
chmod +x hello.sh
```

4. Run it

```
</> Bash
./hello.sh
```


2. Reading User Input

Code:

```
#!/bin/bash
read -p "Enter your name: " name
echo "Hello $name"
```

Execution:

```
chmod +x input.sh
./input.sh
```

Output:

```
Enter your name: Ali
Hello Ali
```

Explanation: Uses `read` to get input from user.

■ SCRIPT 2: Reading User Input

💡 Concept First: What is `read` ?

`read` pauses the script and waits for you to type something, then stores it in a variable.

🔍 Code Breakdown

bash

```
read -p "Enter your name: " name
```

`-p` means prompt — show this message to the user. Whatever the user types gets stored in `name`.

bash

```
echo "Hello $name"
```

Prints hello + whatever the user typed.

Code:

```
#!/bin/bash
read -p "Enter file name: " file
if [ -f "$file" ]; then
    echo "File exists"
else
    echo "File does not exist"
fi
```

Execution:

```
chmod +x filecheck.sh
./filecheck.sh
```

Output:

```
Enter file name: file1.txt
File exists
```

Explanation: Checks if a file exists using if/else.

In bash:

```
bash
if [ condition ]; then
    # do this
else
    # do that
fi
```

⚠️ **fi** = "end of if" (if spelled backwards). Always close your if with **fi** !

Code Breakdown

bash

```
read -p "Enter file name: " file
```

User types a filename, stored in `file`

bash

```
if [ -f "$file" ]; then
```

`-f` means "is this a regular file?" The `[ ]` brackets = a test/condition Always put `"$file"` in quotes — protects against filenames with spaces

bash

```
    echo "File exists"
else
    echo "File does not exist"
fi
```

Simple yes/no output, then `fi` closes the block.

Common `-f` style flags (exam important!)

Flag	Meaning
<code>-f</code>	Is it a file?
<code>-d</code>	Is it a directory?
<code>-e</code>	Does it exist? (file or dir)
<code>-r</code>	Is it readable?

4. Loop Example - List .txt Files

Code:

```
#!/bin/bash
read -p "Enter directory: " dir
for file in "$dir"/*.txt; do
    if [ -f "$file" ]; then
        echo "$file"
    fi
done
```

Execution:

```
chmod +x listtxt.sh
./listtxt.sh
```

Output:

```
/home/user/data/file1.txt
/home/user/data/file2.txt
```

SCRIPT 4: Loop — List `.txt` Files

🧠 Concept First: What is a For Loop?

A loop repeats something for each item in a list. Like:

"For every student in class → take attendance"

```
bash

for variable in list; do
    # do something
done
```

🔍 Code Breakdown

```
bash

for file in "$dir"/*.txt; do
```

`*.txt` = wildcard — means "every file ending in `.txt`" Each `.txt` file gets stored one-by-one in `file`

```
    if [ -f "$file" ]; then
        echo "$file"
    fi
done
```

Double checks it's actually a file (not a folder), then prints its name. `done` closes the loop — like `fi` closes `if`.

5. Function Example - Greeting

Code:

```
#!/bin/bash
greet() {
    echo "Hello $1"
}
greet "Student"
```

Execution:

```
chmod +x greet.sh
./greet.sh
```

Output:

Hello Student

Explanation: Demonstrates a function with argument.

Code Breakdown

```
bash
greet() {
    echo "Hello $1"
}
```

Defines a function called `greet`. `$1` = first argument passed to the function (like a parameter).

```
bash
greet "Student"
```

Calls the function. `"Student"` becomes `$1`. Output → `Hello Student`

Arguments Quick Reference

Symbol	Meaning
<code>\$1</code>	1st argument passed
<code>\$2</code>	2nd argument
<code>\$0</code>	Script name itself
<code>\$@</code>	All arguments

6. File Size Checker Function

Code:

```
#!/bin/bash
check_size() {
    size=$(stat -c%s "$1")
    if [ $size -gt 1048576 ]; then
        echo "Large file: $1"
    else
        echo "File $1 is OK"
    fi
}

read -p "Enter directory: " dir
for file in "$dir"/*; do
    if [ -f "$file" ]; then
        check_size "$file"
    fi
done
```

Execution:

```
chmod +x filesize.sh
./filesize.sh
```

Output:

```
File /home/user/data/file1.txt is OK
Large file: /home/user/data/video.mp4
```

`$(...)` = **command substitution** — run a command and store its output in a variable

`stat -c%s` = get file size **in bytes**

■ SCRIPT 6: File Size Checker

🔍 Code Breakdown

bash

```
check_size() {  
    size=$(stat -c%s "$1")
```

`stat -c%s` = get file size in bytes `$(...)` = command substitution — run a command and store its output in a variable

bash

```
if [ $size -gt 1048576 ]; then
```

`-gt` means greater than `1048576` bytes = 1 MB So: if file is bigger than 1MB → "Large file"

📊 Comparison Operators (very exam important!)

Operator	Meaning
<code>-eq</code>	Equal to
<code>-ne</code>	Not equal
<code>-gt</code>	Greater than
<code>-lt</code>	Less than
<code>-ge</code>	Greater than or equal
<code>-le</code>	Less than or equal

```
read -p "Enter directory: " dir  
for file in "$dir"/*; do  
    if [ -f "$file" ]; then  
        check_size "$file"  
    fi  
done
```

7. File Renaming Utility

Code:

```
#!/bin/bash
dir=$1
pattern=$2
count=1
for file in "$dir"/*; do
    if [ -f "$file" ]; then
        ext="${file##*.}"
        mv "$file" "$dir/$pattern$count.$ext"
        echo "Renamed $file -> $dir/$pattern$count.$ext"
        count=$((count+1))
    fi
done
```

Execution:

```
chmod +x rename_files.sh
./rename_files.sh /home/user/data file
```

Explanation: Renames files sequentially, keeping extension.

 **SCRIPT 7: File Renaming Utility**

 **Code Breakdown**

```
bash

dir=$1
pattern=$2
```

\$1, \$2 here are command-line arguments (passed when running the script)

Example: `./rename.sh /home/user/data file` → `dir=/home/user/data`,
`pattern=file`

`##*.` means: "remove everything up to and including the last dot"

`ext="${file##*.}"`

This **extracts the extension** from a filename

`mv` = rename/move the file `$(( ))` = **math in bash**. Adds 1 to count each loop.

◆ The line:

```
</> Bash
mv "$file" "$dir/$pattern$count.$ext"
count=$((count+1))
```

◆ What is `count` and why we use it?

👉 `count` is just a **number variable** (like a counter)

Purpose:

- To give **unique names** to files
- So files don't overwrite each other

⬆ Get Plus ×

◆ Example setup

Assume:

```
</> Bash
dir=/home/user/data
pattern=file
count=1
ext=txt
```

And current file:

```
</> Bash
file=abc.txt
```

◆ What happens?

```
</> Bash
mv "$file" "$dir/$pattern$count.$ext"
```

Becomes:

```
</> Bash
mv "abc.txt" "/home/user/data/file1.txt"
```

👉 So:

- `abc.txt` is renamed to → `file1.txt`

◆ What are `$1`, `$2`?

In Bash:

- `$1` = first argument
- `$2` = second argument

These are values you type after the script name when running it.

◆ Your code:

```
</> Bash
dir=$1
pattern=$2
```

👉 This means:

- whatever user gives as first input → goes into `dir`
- whatever user gives as second input → goes into `pattern`

◆ Example given:

```
</> Bash
./rename.sh /home/user/data file
```

Breakdown:

- `./rename.sh` → script name
- `/home/user/data` → first argument → `$1`
- `file` → second argument → `$2`

◆ So inside the script:

```
<> Bash
dir=$1      # dir=/home/user/data
pattern=$2  # pattern=file
```

◆ Simple analogy 🍷

Think of:

```
<> Bash
./script arg1 arg2
```

Like:

“Run script and give it 2 inputs”

- `$1` = arg1
- `$2` = arg2



◆ Mini demo script

```
<> Bash
#!/bin/bash
echo "Directory is: $1"
echo "Pattern is: $2"
```

Run:

```
<> Bash
./test.sh myFolder hello
```

Output:

```
Directory is: myFolder
Pattern is: hello
```

`$1`, `$2` mean "arguments", but

Get Plus ×

👉 what they refer to depends on context

◆ 1. In the main script

```
</> Bash
dir=$1
pattern=$2
```



👉 Here:

- `$1`, `$2` = arguments passed to the **script itself**

Example:

```
</> Bash
./script.sh folder file
```



- `$1` = folder
- `$2` = file

◆ 2. Inside a function

```
</> Bash
myFunc() {
  echo $1
  echo $2
}
```



👉 Here:

- `$1`, `$2` = arguments passed to the **function**, NOT the script

◆ Example to make it crystal clear

```
</> Bash
#!/bin/bash

echo "Script arguments: $1 $2"

myFunc() {
    echo "Function arguments: $1 $2"
}

myFunc hello world
```

◆ Run:

```
</> Bash
./script.sh A B
```

◆ Output:

```
Script arguments: A B
Function arguments: hello world
```

8. Directory Cleanup Script

Code:

```
#!/bin/bash
dir=$1
days=$2
count=$(find "$dir" -type f -mtime +$days | wc -l)
find "$dir" -type f -mtime +$days -exec rm -v {} \;
find "$dir" -type d -empty -delete
echo "Deleted $count files older than $days days"
```

Execution:

```
chmod +x cleanup.sh
./cleanup.sh /home/user/data 30
```

Explanation: Deletes files older than N days and removes empty directories.

Explanation: Deletes files older than N days and removes empty directories.

SCRIPT 8: Directory Cleanup

Code Breakdown

bash

```
count=$(find "$dir" -type f -mtime +$days | wc -l)
```

`find` = search for files `-type f` = only files (not folders) `-mtime +30` = files older than 30 days `wc -l` = count the lines (i.e., count the files)

Find = search for files . `-type f` means only files not folders

`-mtime +$days` means files older than specific days

`Wc -l` means count the lines (ie count the files)

bash

```
find "$dir" -type f -mtime +$days -exec rm -v {} \;
```

`-exec rm -v {} \;` = execute `rm` (delete) on each found file `{}` = placeholder for each file found `-v` = verbose (show what's being deleted)

bash

```
find "$dir" -type d -empty -delete
```

`-type d` = directories only `-empty` = only empty ones `-delete` = remove them

```
find "$dir" -type f -mtime +$days -exec rm -v {} \;  
find "$dir" -type d -empty -delete
```

9. Backup Script

Code:

```
#!/bin/bash
read -p "Enter directory to backup: " dir
backup_name="backup_$(date +%Y%m%d).tar.gz"
tar -czf "$backup_name" -C "$dir" .
echo "Backup completed: $backup_name" >> backup.log
```

Execution:

Execution:

```
chmod +x backup.sh
./backup.sh
```

Explanation: Creates date-stamped backup and logs the action.

❖ Script 1 - Backup script

🔗 Code Breakdown

bash

```
backup_name="backup_$(date +%Y%m%d).tar.gz"
```

`date +%Y%m%d` = today's date like `20250328` So backup file becomes:

```
backup_20250328.tar.gz
```

bash

```
tar -czf "$backup_name" -C "$dir" .
```

`tar` = archive tool `-c` = create, `-z` = compress (gzip), `-f` = filename `-C "$dir" .`
= go into that directory and archive everything

bash

```
echo "Backup completed: $backup_name" >> backup.log
```

`>>` = append to file (doesn't overwrite, adds to end) `>` would overwrite — know this difference!

🔗 What actually happens

You are NOT writing first and then doing something.

👉 Instead, the flow is:

1. Backup is created first
2. Then a message about it is written to `backup.log`

🔗 Your script order:

🔗 Bash

```
tar -czf "$backup_name" -C "$dir" .  
echo "Backup completed: $backup_name" >> backup.log
```


10. System Monitoring Script (Mini Example)

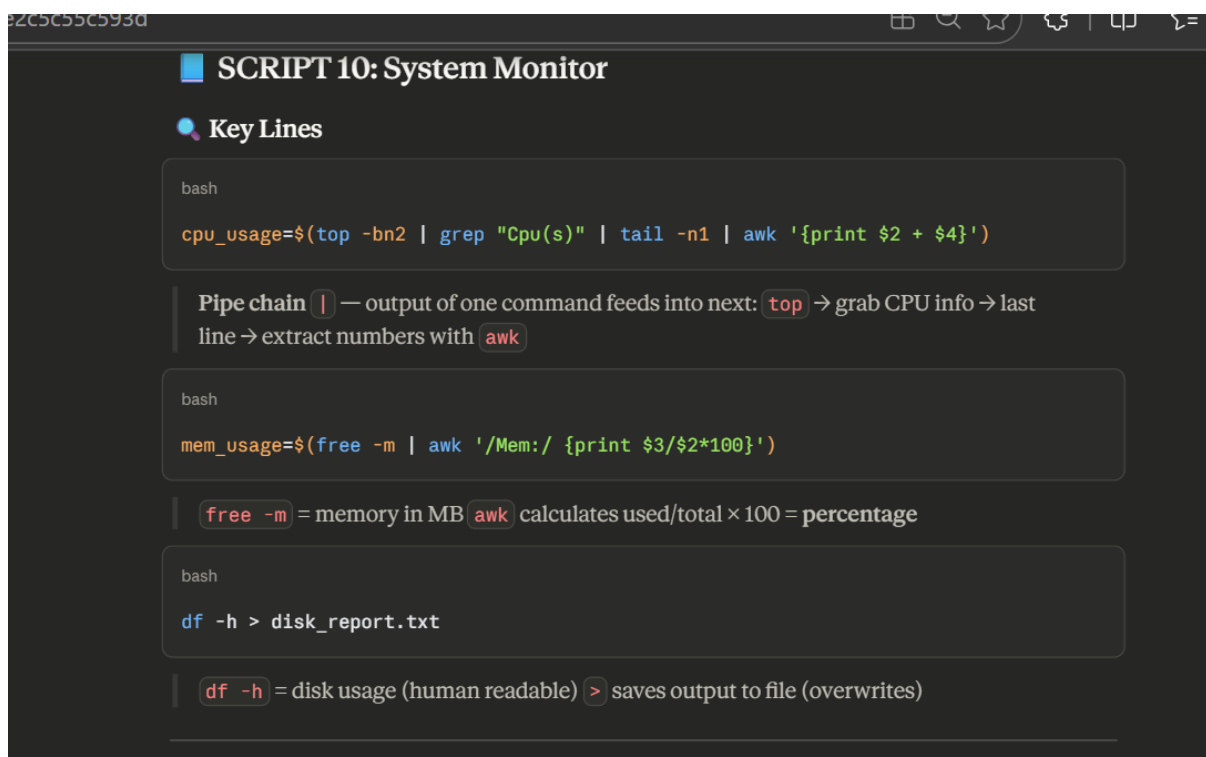
Code:

```
#!/bin/bash
read -p "Monitoring duration in seconds: " duration
cpu_usage=$(top -bn2 | grep "Cpu(s)" | tail -n1 | awk '{print $2 + $4}')
mem_usage=$(free -m | awk '/Mem:/ {print $3/$2*100}')
df -h > disk_report.txt
echo "CPU Usage: $cpu_usage%" > system_report.txt
echo "Memory Usage: $mem_usage%" >> system_report.txt
echo "Disk usage report saved in disk_report.txt" >> system_report.txt
```

Execution:

```
chmod +x monitor.sh
./monitor.sh
```

Explanation: Collects CPU, memory, disk info and saves report.



SCRIPT 10: System Monitor

Key Lines

```
bash
cpu_usage=$(top -bn2 | grep "Cpu(s)" | tail -n1 | awk '{print $2 + $4}')
```

Pipe chain | — output of one command feeds into next: `top` → grab CPU info → last line → extract numbers with `awk`

```
bash
mem_usage=$(free -m | awk '/Mem:/ {print $3/$2*100}')
```

`free -m` = memory in MB `awk` calculates $\text{used}/\text{total} \times 100 = \text{percentage}$

```
bash
df -h > disk_report.txt
```

`df -h` = disk usage (human readable) `>` saves output to file (overwrites)

◆ Example Run

✦ Get Plus ✕

Step 1: Run script

```
</> Bash  
./monitor.sh
```



Step 2: Input

```
</> Bash  
Monitoring duration in seconds: 5
```



◆ What happens internally

✓ CPU usage (example)

```
</> Bash  
cpu_usage = 12.5
```



✓ Memory usage (example)



```
</> Bash  
mem_usage = 43.2
```



e-1184cec99a09



◆ Files created

✦ Get Plus ✕

 system_report.txt

```
CPU Usage: 12.5%  
Memory Usage: 43.2%  
Disk usage report saved in disk_report.txt
```



 disk_report.txt

```
Filesystem      Size  Used Avail Use% Mounted on  
/dev/sda1       50G   25G   23G   52% /  
tmpfs           1.9G  200M   1.7G   10% /run
```



11. Moving Files Based on Extension

Code:

```
#!/bin/bash
read -p "Enter directory: " dir
for file in "$dir"/*; do
    if [ -f "$file" ]; then
        ext="${file##*.}"
        mkdir -p "$dir/$ext"

        mv "$file" "$dir/$ext/"
        echo "Moved $file -> $dir/$ext/"
    fi
done
```

Execution:

```
chmod +x move_ext.sh
./move_ext.sh
```

Explanation: Organizes files into folders by extension.

■ SCRIPT 11: Move Files by Extension

🔑 Key Line

```
bash
ext="${file##*.}"
mkdir -p "$dir/$ext"
mv "$file" "$dir/$ext/"
```

Extract extension → create folder for it → move file into it `-p` in `mkdir` = don't error if folder already exists

12. Appending Logs with Date/Time

Code:

```
#!/bin/bash
echo "Script run at $(date)" >> run_log.txt
```

Execution:

```
chmod +x log_append.sh
./log_append.sh
```

Explanation: Appends date/time to a log file every time script runs.

■

SCRIPT 12: Append Log with Date

```
bash
echo "Script run at $(date)" >> run_log.txt
```

Simplest logging pattern. Every run appends a timestamped line.

● Category 6: Power Commands (appear in complex scripts)

Command	Meaning
stat -c%s file	Get file size in bytes
date +%Y%m%d	Today's date as 20250328
tar -czf name.tar.gz	Create compressed archive
free -m	Show memory usage in MB
df -h	Show disk usage (human readable)
awk '{print \$2}'	Extract column 2 from output
grep "pattern"	Search for text in output
"\${file##*.}"	Extract file extension

📁 The 12 Scripts — Layer 1 Summary (memorize these sentences!)

#	Script	One Line Summary
1	Hello World	Store name in variable, print it
2	User Input	<code>read</code> user's name, greet them
3	File Check	If file exists → say so, else → say not found
4	List .txt files	Loop through dir, print <code>.txt</code> files
5	Function Greet	Define function, pass name as <code>\$1</code> , print it
6	File Size Check	Loop files, use <code>stat</code> to check if > 1MB
7	File Rename	Loop files, extract ext, rename with pattern+count
8	Directory Cleanup	<code>find</code> old files → delete them → remove empty dirs
9	Backup	<code>tar</code> a directory → name with date → log it
10	System Monitor	Get CPU/mem/disk → save to report file
11	Move by Extension	Extract ext → make folder for it → move file in
12	Log Append	<code>echo</code> date+time <code>>></code> log file

Practise

Q2. [=10wtg 20 marks]

Part (A)

Design a file-copying program named `filecopy.c` using ordinary pipes. This program will be passed two parameters: the name of the file to be copied and the name of the destination file. The program will create an ordinary pipe and write the contents of the file to be copied to the pipe. The child process will read this file from the pipe and write it to the destination file. For example, if we invoke the program as follows:

- `./filecopy input.txt copy.txt`
- The file `input.txt` will be written to the pipe. The child process will read the contents of this file and write it to the destination file `copy.txt`.

...

✓ Complete Program

```
c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define BUFFER_SIZE 1024
#define READ_END 0
#define WRITE_END 1

int main(int argc, char *argv[]) {

    // — Check arguments —
    if (argc != 3) {
        printf("Usage: ./filecopy source.txt dest.txt\n");
        exit(1);
    }
```

```
        exit(1);
    }

    int fd[2];
    pid_t pid;
    char buffer[BUFFER_SIZE];
    int bytes;

    // — Create pipe —
    if (pipe(fd) == -1) {
        perror("pipe");
        exit(1);
    }
```

```
pid = fork();
if (pid < 0) {
    perror("fork");
    exit(1);
}

if (pid > 0) {
    // — PARENT: read source, write to pipe —
    close(fd[READ_END]); // parent doesn't read

    // open source file
    int src = open(argv[1], O_RDONLY);
    if (src < 0) {
        perror("open source");
        exit(1);
    }
}
```

```
    }

    // read source and write into pipe
    while ((bytes = read(src, buffer, BUFFER_SIZE)) > 0) {
        write(fd[WRITE_END], buffer, bytes);
    }

    close(src);
    close(fd[WRITE_END]); // done writing to pipe
    wait(NULL);           // wait for child
    printf("Parent: File copied successfully!\n");
}
```

```

        printf("Parent: File copied successfully!\n");
    }
    else {
        // — CHILD: read from pipe, write to dest —
        close(fd[WRITE_END]); // child doesn't write

        // open destination file
        int dst = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (dst < 0) {
            perror("open dest");
            exit(1);
        }

        // read from pipe and write to destination
        while ((bytes = read(fd[READ_END], buffer, BUFFER_SIZE)) > 0) {
            write(dst, buffer, bytes);
        }

        close(dst);
        close(fd[READ_END]); // done reading from pipe
        printf("Child: Written to %s successfully!\n", argv[2]);
        exit(0);
    }

    return 0;
}

```

Compile & Run

```

bash

gcc filecopy.c -o filecopy

# create a test file
echo "Hello this is test content!" > input.txt

# run it
./filecopy input.txt copy.txt

# verify
cat copy.txt
...

---

# Expected Output
...
Child: Written to copy.txt successfully!
Parent: File copied successfully!
...

```


SYSTEM CALLS:

🧠 FIRST — The Big Picture (Most Important Concept)

Before any code, understand **this one idea**:

Your C program → talks to → The OS Kernel → controls → Hardware

Normally your C program does things on its own. But some things — like **creating a new process**, **reading a file**, **sleeping** — only the OS kernel can do.

So you make a **system call** — basically saying:

"Hey OS, I need your help to do this."

That's it. System calls = **asking the OS for help**.

📋 The Standard Template (Memorize This!)

Every single system call program follows this skeleton:

```
c
#include <stdio.h>      // printf, scanf
#include <stdlib.h>     // exit(), EXIT_SUCCESS
#include <unistd.h>     // fork(), getpid(), sleep(), exec()
#include <sys/types.h>  // pid_t (process ID type)
#include <sys/wait.h>   // wait()
#include <errno.h>      // error handling

int main() {
    // 1. Declare variables
    // 2. Make system calls
    // 3. Handle errors
    return 0;
}
```

🟦 CATEGORY 1: Process Management

🧠 Concept: What is a Process?

A **process** is a running program.

Right now, your browser, your terminal, your music app — each one is a **separate process**. Every process has a unique ID number called a **PID** (Process ID).

★ System Call 1: `fork()`

Plain English:

`fork()` = make a copy of yourself

When your program calls `fork()`, the OS **duplicates** your process. Now there are **two processes** running — the original (parent) and the copy (child). Both continue from the **very next line** after `fork()`.

How do you tell them apart?

`fork()` returns a different value to each:

```
Parent gets → child's PID (a number > 0)
Child gets  → 0
Error       → -1
```

Think of it like this:

```
Before fork():  [Parent Process]
                  |
                fork()
                 /  \
After fork():  [Parent] [Child]
               gets >0 gets 0
```

C

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;          // pid_t is just a special int for process IDs

    pid = fork();        // create child process

    if (pid < 0) {
        // fork failed
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0) {
        // I am the CHILD (fork returned 0 to child)
        printf("I am the CHILD. My PID = %d\n", getpid());
    }
    else {
        // I am the PARENT (fork returned child's PID to parent)
        printf("I am the PARENT. My PID = %d\n", getpid());
        printf("My child's PID = %d\n", pid);
    }

    return 0;
}
```

Line by Line:

Line	Meaning
<code>pid_t pid</code>	Declare a variable to store the process ID
<code>pid = fork()</code>	Create child. Return value stored in <code>pid</code>
<code>if (pid < 0)</code>	Fork failed — always check this
<code>if (pid == 0)</code>	This block runs ONLY in child
<code>else</code>	This block runs ONLY in parent
<code>getpid()</code>	Returns current process's own PID

★ System Call 2: `wait()`

Plain English:

`wait()` = parent pauses and waits for child to finish

Without `wait()`, the parent might finish and exit before the child is done. That creates a **zombie process** (child with no parent). Always use `wait()` in parent!

c



```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // CHILD
        printf("Child working...\n");
        sleep(2);           // child sleeps 2 seconds
        printf("Child done!\n");
    }
    else {
        // PARENT
        printf("Parent waiting...\n");
        wait(NULL);         // NULL = we don't care about exit status
        printf("Parent done! Child has finished.\n");
    }

    return 0;
}

```

📌 `wait(NULL)` — the `NULL` just means "I don't need the child's exit code, just wait for it."



★ System Call 3: `exec()`

Plain English:

`exec()` = replace yourself with a different program

The child process **transforms** into a completely different program. The original code is gone — it's now running something else (like `ls`, `pwd`, etc.)

The version you'll use most: `execvp()`

```

c
execvp("ls", "ls", NULL);
//      ^      ^      ^
//   program arg0 end marker

```

Example — child runs `ls`:

```
c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // CHILD — replace itself with "ls" command
        execlp("ls", "ls", NULL);
        // if execlp succeeds, lines below NEVER run
        printf("This won't print if exec succeeded\n");
    }
    else {
        // PARENT — wait for child
        wait(NULL);
        printf("Child finished running ls\n");
    }

    return 0;
}
```

🔑 **Key rule:** If `execlp()` works, everything after it in the child is ignored. The child is now a different program entirely.

So in your example:

```
</> C
execlp("ls", "ls", NULL);
```

Argument	Meaning
"ls" (first)	the name of the executable to run (finds it in <code>\$PATH</code>)
"ls" (second)	the <code>argv[0]</code> of the program — usually same as program name
<code>NULL</code>	marks the end of arguments

✅ Essentially, this runs the `ls` command (lists files in the current directory).

3 Why `"ls"` appears twice

- First `"ls"` → tells `execlp()` which program to ↓ and run
- Second `"ls"` → tells the program what `argv[0]` is
 - Every program gets `argv[0]` as its name
 - In C, `int main(int argc, char *argv[])` → here `argv[0] = "ls"`

★ System Call 4: `exit()`

Plain English:

`exit()` = stop this process right now

```
c
exit(0); // EXIT_SUCCESS - everything went fine
exit(1); // EXIT_FAILURE - something went wrong
```

Usually used in the **child** to cleanly terminate:

```
c
if (pid == 0) {
    printf("Child done\n");
    exit(0); // child exits cleanly
}
```

● CATEGORY 2: Information Retrieval

These are the simple ones — no confusion here.

`getpid()` and `getppid()`

```
c
printf("My PID = %d\n", getpid()); // current process ID
printf("My Parent PID = %d\n", getppid()); // parent's ID
```

↗ C

```
alarm(5); // send SIGALRM signal after 5 seconds - kills program
```

Think of `alarm()` as a **countdown bomb** 💣 — after N seconds, it sends a `SIGALRM` signal to the process, which kills the program unless the signal is handled.

CATEGORY 3: File Management – File Descriptors

🧠 Concept Recap: File Descriptor

- A **file descriptor (FD)** is just an **integer** representing an open file
- Standard FDs:

FD	Name	Purpose
0	stdin	keyboard input
1	stdout	screen output
2	stderr	error messages
3+	custom	files you open

★ System Calls for File Handling

1 open()

```
</> C
#include <fcntl.h>
int fd = open("file1.txt", O_RDONLY); // open for reading
```

- Returns a **file descriptor** (`int`)

Flag	Meaning
O_RDONLY	open file for reading
O_WRONLY	open file for writing
O_RDWR	read & write
O_CREAT	create file if not exist
O_TRUNC	truncate file to 0 bytes

```
int fd = open("file.txt", O_WRONLY | O_CREAT, 0644); // open for writing
```

Flag	Meaning
------	---------

| `0644` = file permissions (owner can read/write, others can read)

2. `read()` — Read from file

```
c
char buffer[100];
int bytes = read(fd, buffer, sizeof(buffer));
```

Read from `fd`, store into `buffer`, read up to `sizeof(buffer)` bytes.

3. `write()` — Write to file

```
c
write(fd, buffer, bytes);
```

Write `bytes` number of bytes from `buffer` into `fd`.

4. `close()` — Close the file

```
c
close(fd);
```

Always close when done. Like closing a book after reading.

1 What `read()` does

```
</> C
bytes_read = read(src, buffer, sizeof(buffer));
```

- `src` → file descriptor of `input.txt`
- `buffer` → where the data will be stored
- `sizeof(buffer)` → max bytes to read at once (here 1024)

Important:

- `read()` does not guarantee it reads the whole file at once
- It reads up to 1024 bytes at a time (or less if end-of-file is near)
- Returns number of bytes actually read
- At the end of the file → `read()` returns `0`
- On error → returns `-1`

2 Why we need a `while` loop

- Files can be larger than 1024 bytes
- We cannot read the whole file in a single call
- So we keep reading chunks in a loop until the file ends

PRACTISE QUESTIONS

Full File Copy Example (maps to In-Lab Q5):

```
c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int src, dst;
    char buffer[1024];
    int bytes_read;

    // Open source file for reading
    src = open("input.txt", O_RDONLY);
    if (src < 0) {
        printf("Error opening input.txt\n");
        exit(1);
    }
```

```
    // Open destination file for writing (create if not exists)
    dst = open("output.txt", O_WRONLY | O_CREAT, 0644);
    if (dst < 0) {
        printf("Error opening output.txt\n");
        exit(1);
    }
```

```

// Read from source and write to destination in chunks
bytes_read = read(src, buffer, sizeof(buffer));
while (bytes_read > 0) {
    write(dst, buffer, bytes_read);
    bytes_read = read(src, buffer, sizeof(buffer));
}

close(src);
close(dst);

printf("File copied successfully!\n");
return 0;
}

```

🔗 Master Cheat Sheet — System Calls

Call	What it does	Returns
<code>fork()</code>	Create child process	<code>0</code> to child, <code>>0</code> to parent, <code>-1</code> on error
<code>wait(NULL)</code>	Parent waits for child	Child's PID when done
<code>exit(0)</code>	Terminate process	Nothing
<code>execvp(prog, arg, NULL)</code>	Replace process with new program	<code>-1</code> on failure
<code>getpid()</code>	Get my PID	integer
<code>getppid()</code>	Get parent's PID	integer
<code>sleep(n)</code>	Pause n seconds	—
<code>alarm(n)</code>	Set timer for n seconds	—
<code>open(path, flags)</code>	Open file	file descriptor int
<code>read(fd, buf, size)</code>	Read from file	bytes read
<code>write(fd, buf, size)</code>	Write to file	bytes written
<code>close(fd)</code>	Close file	—



🔗 The Golden Pattern for fork() — Memorize This!

```

c
pid_t pid = fork();

if (pid < 0)      { /* ERROR   - fork failed   */ }
else if (pid == 0){ /* CHILD   - pid is 0       */ }
else             { /* PARENT  - pid is child's */ }

```

In-lab problems:

1. Write a program that creates two child processes. One child process should print its process ID (PID), and the other child process should print its parent process ID (PPID). The parent process should wait for both child processes to terminate before ending. **Hint: fork (), wait ()**

```
[*] Untitled1
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/wait.h>
4
5  int main() {
6      pid_t child1, child2;
7
8      child1 = fork();
9
10     if (child1 == 0) {
11         // First child
12         printf("Child 1 - My PID is: %d\n", getpid());
13         return 0;
14     }
15
16     child2 = fork();
17
18     if (child2 == 0) {
19         // Second child
20         printf("Child 2 - My Parent's PID (PPID) is: %d\n", getppid());
21         return 0;
22     }
23
24     // Parent waits for both children
25     wait(NULL);
26     wait(NULL);
27     printf("Parent: Both children have finished.\n");
28
29     return 0;
30 }
```

Page 1 / 11

2. Create a program that creates a child process. The child process prints "I am a child process" 100 times in a loop. Whereas the parent process prints "I am a parent process" 100 times in a loop.

```
[*] Untitled1
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/wait.h>
4
5  int main() {
6      pid_t pid = fork();
7
8      if (pid == 0) {
9
10         for (int i = 0; i < 100; i++) {
11             printf("I am a child process\n");
12         }
13     } else {
14
15         for (int i = 0; i < 100; i++) {
16             printf("I am a parent process\n");
17         }
18         wait(NULL); // Wait for child to finish
19     }
20
21     return 0;
22 }
```

3. Develop a program that prints the current process ID (PID) and the parent process ID (PPID) of the running process. Additionally, implement a function to retrieve and print the user ID (UID) of the current user executing the program. **Hint: getpid (), getppid ()**

```
[*] Untitled1
1  #include <stdio.h>
2  #include <unistd.h>
3
4  void printUID() {
5      printf("User ID (UID): %d\n", getuid());
6  }
7
8  int main() {
9      printf("Current Process ID (PID) : %d\n", getpid());
10     printf("Parent Process ID (PPID) : %d\n", getppid());
11     printUID();
12
13     return 0;
14 }
```

4. Create a program named stat that takes an integer array as command line argument (delimited by some character such as \$). The program then creates 3 child processes each of which does exactly one task from the following: i) Adds them and print the result on the screen. (done by child 1), ii) Shows the average on the screen. (done by child 2), iii) Prints the maximum number on the screen. (done by child 3)

STEP 1: Understand What The Program Must Do

You run: `./stat 10$20$30$40`

The program reads: `"10$20$30$40"`

Splits it into: `[10, 20, 30, 40]`

Then creates 3 children:

Child 1 → prints Sum = 100

Child 2 → prints Average = 25

Child 3 → prints Maximum = 40

Parent → waits for all 3 children to finish

🧠 STEP 2: New Concepts You Need First

Concept 1: Command Line Arguments

When you run `./stat 10$20$30`, your `main()` receives:

```
c
int main(int argc, char *argv[])
```

Variable	Meaning	Value in our case
<code>argc</code>	argument count	<code>2</code>
<code>argv[0]</code>	program name	<code>"./stat"</code>
<code>argv[1]</code>	first argument	<code>"10\$20\$30"</code>

Concept 2: Splitting a String by `$`

We use `strtok()` — it splits a string by a delimiter.

```
c
char *token = strtok(argv[1], "$"); // split by $
while (token != NULL) {
    printf("%s\n", token);           // prints each number
    token = strtok(NULL, "$");       // get next piece
}
...
```

Think of `'strtok'` like cutting a string at every `'$'`:

`"10$20$30"` → `"10"` `"20"` `"30"`

Concept 3: Forking 3 Children

The pattern for 3 children is:

```
c
fork(); // child 1
fork(); // child 2
fork(); // child 3
```

But this is **WRONG** — it creates way more than 3!

The correct way — fork one at a time, child exits immediately:

But this is **WRONG** — it creates way more than 3!

The correct way — fork one at a time, child exits immediately:

```
c

pid1 = fork();
if (pid1 == 0) { /* child 1 work */ exit(0); }

pid2 = fork();
if (pid2 == 0) { /* child 2 work */ exit(0); }

pid3 = fork();
if (pid3 == 0) { /* child 3 work */ exit(0); }

// parent waits for all 3
wait(NULL);
wait(NULL);
wait(NULL);
```

✦ **Key trick:** Each child calls `exit(0)` immediately after its job. This stops it from going on and creating more forks!

```
c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {

    // — Check usage —————
    if (argc < 2) {
        printf("Usage: ./stat num1$num2$num3\n");
        exit(1);
    }

    // — Parse numbers from "10$20$30$40" —————
    int numbers[100];
    int count = 0;

    char *token = strtok(argv[1], "$");
    while (token != NULL) {
        numbers[count++] = atoi(token);
        token = strtok(NULL, "$");
    }
}
```

```
pid_t pid1, pid2, pid3;

// — CHILD 1: Sum —————
pid1 = fork();
if (pid1 < 0) { printf("Fork 1 failed\n"); exit(1); }
if (pid1 == 0) {
    int sum = 0;
    for (int i = 0; i < count; i++) sum += numbers[i];
    printf("Child 1 (Sum)      = %d\n", sum);
    exit(0);
}
```

```
// — CHILD 2: Average —————
pid2 = fork();
if (pid2 < 0) { printf("Fork 2 failed\n"); exit(1); }
if (pid2 == 0) {
    int sum = 0;
    for (int i = 0; i < count; i++) sum += numbers[i];
    float avg = (float)sum / count;
    printf("Child 2 (Average) = %.2f\n", avg);
    exit(0);
}
```

```
// — CHILD 3: Maximum —————
pid3 = fork();
if (pid3 < 0) { printf("Fork 3 failed\n"); exit(1); }
if (pid3 == 0) {
    int max = numbers[0];
    for (int i = 1; i < count; i++)
        if (numbers[i] > max) max = numbers[i];
    printf("Child 3 (Maximum) = %d\n", max);
    exit(0);
}
```



```

// — PARENT: Wait for all 3 —————
wait(NULL);
wait(NULL);
wait(NULL);
printf("Parent: All children finished!\n");

return 0;
}

```

▶ How to Compile & Run

```

bash

gcc stat.c -o stat      # compile
./stat 10$20$30$40     # run with arguments
...

```

Expected Output:

```

...
Child 1 (Sum)      = 100
Child 2 (Average) = 25.00
Child 3 (Maximum) = 40
Parent: All children finished!

```

⚠ Output order of children may vary — that's normal! OS decides which child runs first.

💡 The 5 Things To Remember From This Program

Thing	Why Important
<code>int main(int argc, char *argv[])</code>	How to accept command line args
<code>strtok(str, "\$")</code> then <code>strtok(NULL, "\$")</code>	How to split string by delimiter
<code>atoi(token)</code>	Convert string <code>"10"</code> to integer <code>10</code>
<code>exit(0)</code> after each child	Stops child from forking more children
Three <code>wait(NULL)</code> in parent	One <code>wait</code> per child — always match them

- Invoke at least 4 commands from your programs, such as `Cp`, `mkdir`, `rmdir`, etc (The calling program must not be destroyed)

The Key Concept Here

"The calling program must not be destroyed"

This is the crucial part. Remember `exec()` replaces the current process. So if parent calls `exec()` directly — parent is gone! ❌

The solution: Always `fork()` first, then let the child call `exec()`. Parent stays safe. ✅

Parent forks → Child runs exec (child gets replaced)
Parent is safe → waits → continues to next command

The Pattern For Each Command

Every single command follows this exact same pattern:

```
c
pid = fork();
if (pid == 0) {
    execlp("command", "command", args, NULL);
    exit(0); // safety exit if exec fails
}
wait(NULL); // parent waits for child to finish
```

Just repeat this block for each command. That's it!

That's simply the **argument** you're passing — same as typing it in terminal:

```
bash

# Terminal:
mkdir myFolder

# execlp equivalent:
execlp("mkdir", "mkdir", "myFolder", NULL);
...

---

# Quick Pattern For Any Command
...

terminal:  command  arg1    arg2
execlp:    "command", "command", "arg1", "arg2", NULL
```

`execvp()` Structure Explained

```
c
execvp("mkdir", "mkdir", "myFolder", NULL);
//      [1]      [2]      [3]      [4]
```

Position	What it is	Plain English
[1] "mkdir"	Program to find	Go find a program called <code>mkdir</code> in the system
[2] "mkdir"	argv[0]	The name the program calls itself
[3] "myFolder"	argv[1]	The argument passed to mkdir
[4] NULL	End marker	Stop here, no more arguments

So for example:

```
c
// cp file1.txt file2.txt
execvp("cp", "cp", "file1.txt", "file2.txt", NULL);

// ls -l
execvp("ls", "ls", "-l", NULL);

// rmdir myFolder
execvp("rmdir", "rmdir", "myFolder", NULL);
```

Simple rule → whatever you type in terminal, just wrap each word in quotes and add NULL at the end! 😊

Running Your Own Program with `execlp`

Same structure! You just give it the path to your program instead.

```
c
execlp("./myprogram", "myprogram", NULL);
//      ^           ^
//  path to file   argv[0]
```

Examples

```
c
// Run your own program with no arguments
execlp("./hello", "hello", NULL);

// Run your own program WITH arguments
execlp("./stat", "stat", "10$20$30", NULL);
```

```
c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    pid_t pid;

    printf("Parent PID = %d\n\n", getpid());

    // — COMMAND 1: mkdir - create a directory —
    printf("--- Running: mkdir myFolder ---\n");
    pid = fork();
    if (pid < 0) { printf("Fork failed\n"); exit(1); }
    if (pid == 0) {
        execlp("mkdir", "mkdir", "myFolder", NULL);
        exit(0); // only runs if execlp fails
    }
    wait(NULL); // parent waits for child
    printf("mkdir done!\n\n");
}
```

We are having only one child working so do wait (NULL); after each thing

```
// — COMMAND 2: ls - list directory contents —  
printf("--- Running: ls ---\n");  
pid = fork();  
if (pid < 0) { printf("Fork failed\n"); exit(1); }  
if (pid == 0) {  
    execlp("ls", "ls", NULL);  
    exit(0);  
}  
wait(NULL);  
printf("ls done!\n\n");
```

```
// — COMMAND 3: cp - copy a file —————  
// creates a copy of this source file  
printf("--- Running: cp ---\n");  
pid = fork();  
if (pid < 0) { printf("Fork failed\n"); exit(1); }  
if (pid == 0) {  
    execlp("cp", "cp", "invoke.c", "myFolder/invoke_copy.c", NULL);  
    exit(0);  
}  
wait(NULL);  
printf("cp done!\n\n");
```

```
// — COMMAND 4: rmdir - remove the directory —  
printf("--- Running: rmdir myFolder ---\n");  
pid = fork();  
if (pid < 0) { printf("Fork failed\n"); exit(1); }  
if (pid == 0) {  
    execlp("rmdir", "rmdir", "myFolder", NULL);  
    exit(0);  
}  
wait(NULL);  
printf("rmdir done!\n\n");
```

```

// — Parent is still alive and running! —————
printf("Parent PID = %d still alive!\n", getpid());
printf("All 4 commands executed successfully!\n");

return 0;
}

```

```
gcc invoke.c -o invoke
```

```
./invoke
```

```
...
```

```
### Expected Output:
```

```
...
```

```
Parent PID = 1234
```

```
--- Running: mkdir myFolder ---
```

```
mkdir done!
```

```
--- Running: ls ---
```

```
invoke invoke.c myFolder
```

```
ls done!
```

```
--- Running: cp ---
```

```
cp done!
```

```
--- Running: rmdir myFolder ---
```

```
rmdir done!
```

```
Parent PID = 1234 still alive!
```

```
All 4 commands executed successfully!
```

✦ Notice Parent PID at the start and end is  same number — proof the parent was never destroyed!

Reply...

- Write a program that demonstrates the usage of the `fork()` system call to create a child process. The child process should execute a shell command to list all files in the current directory, while the parent process waits for the child to terminate. Ensure proper synchronization between parent and child processes. **Hint: `fork()`, `execlp()`**

✓ Complete Program

c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```
int main() {
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    pid = fork();
```

```
    if (pid < 0) {
```

```
        // fork failed
```

```
        printf("Fork failed!\n");
```

```
        exit(1);
```

```
    }
```

```
    else if (pid == 0) {
```

```
        // CHILD - list all files in current directory
```

```
        printf("Child (PID=%d): listing files...\n\n", getpid());
```

```
        execlp("ls", "ls", NULL);
```

```
        // only reaches here if execlp failed
```

```
        printf("execlp failed!\n");
```

```
        exit(1);
```

```
    }
```

```
    else {
```

```
        exit(1);
```

```
    }
```

```
    else {
```

```
        // PARENT - wait for child to finish
```

```
        printf("Parent (PID=%d): waiting for child...\n", getpid());
```

```
        wait(NULL);
```

```
        printf("\nParent: child has terminated. Done!\n");
```

```
    }
```

```
    return 0;
```

```
    }
```

```
    ...
```

```
    ---
```

8. Implement a program that utilizes the `sleep()` and `alarm()` system calls. The program should initially set an alarm for 5 seconds and then enter a sleep loop. Within this loop, the program should print a message every second. When the alarm signal is received after 5 seconds, the program should terminate gracefully. **Hint: `sleep()`, `alarm()`**

New Concept: Signal Handling

Remember `alarm(5)` sends a signal called `SIGALRM` after 5 seconds.

By default `SIGALRM` just kills the program — not "graceful."

Graceful termination means:

"Before dying, print a goodbye message then exit cleanly"

To do that we need to **catch** the signal and run our own function instead. This is called a **Signal Handler**.

`alarm(5)` fires → `SIGALRM` sent → our function runs → we exit cleanly

How To Write a Signal Handler

```
c
#include <signal.h> // new header needed!

// Step 1: Write the handler function
void myHandler(int sig) {
    printf("Signal received! Goodbye!\n");
    exit(0);
}

// Step 2: Register it - tell OS "use MY function for SIGALRM"
signal(SIGALRM, myHandler);
```

That's it! Two steps — write function, register it.


```

}

int main() {

    // Step 1: Register our handler for SIGALRM
    signal(SIGALRM, alarmHandler);

    // Step 2: Set alarm for 5 seconds
    alarm(5);
    printf("Alarm set for 5 seconds!\n\n");
}

```

```

int main() {

    // Step 1: Register our handler for SIGALRM
    signal(SIGALRM, alarmHandler);

    // Step 2: Set alarm for 5 seconds
    alarm(5);
    printf("Alarm set for 5 seconds!\n\n");

    // Step 3: Loop - print message every second
    int seconds = 0;
    while (1) {
        seconds++;
        printf("Running... %d second(s) passed\n", seconds);
        sleep(1); // wait 1 second, then loop again
    }

    // Never reaches here - alarm kills it at 5 seconds
    return 0;
}
...

```

2. Write a program which prints its PID and uses fork () system call to create a child process. After fork () system call, both parent and child processes print what kind of process they are and their PID. Also, the parent process prints its child's PID, and the child process prints its parent's PID.

✓ Complete Program

```
c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    pid_t pid;

    // — Before fork — print program's own PID —
    printf("Program PID = %d\n\n", getpid());

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        exit(1);
    }
    else if (pid == 0) {
        // CHILD
        // pid == 0 here, so to get parent's PID use getppid()
        printf("I am the CHILD. My PID = %d, My Parent's PID = %d\n",
              getpid(), getppid());
    }
    else {
        // PARENT
        // pid here = child's PID (returned by fork)
        printf("I am the PARENT. My PID = %d, My Child's PID = %d\n",
              getpid(), pid);
        wait(NULL);
    }

    return 0;
}
...
```

We get child pid by fork in parent

3. Develop a program that copies the contents of one file into another file using system calls. The program should accept two file paths as command-line arguments: the source file to be copied from and the destination file to be copied to. Ensure proper error handling for file opening, reading, and writing operations.

✓ Complete Program

```
c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>      // for open() flags: O_RDONLY, O_WRONLY etc

int main(int argc, char *argv[]) {

    // — Check correct usage —————
    if (argc != 3) {
        printf("Usage: ./copy source.txt destination.txt\n");
        exit(1);
    }

    int src, dst;
    char buffer[1024];
    int bytes_read;

    // — Step 1: Open source file for reading ———
    src = open(argv[1], O_RDONLY);
    if (src < 0) {
        printf("Error: Cannot open source file '%s'\n", argv[1]);
        exit(1);
    }
}
```

```

// — Step 2: Open destination file for writing —
// O_WRONLY = write only
// O_CREAT  = create if doesn't exist
// O_TRUNC  = if file exists, clear it first
// 0644     = permissions: owner read/write, others read
dst = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (dst < 0) {
    printf("Error: Cannot open destination file '%s'\n", argv[2]);
    close(src);    // close source before exiting!
    exit(1);
}

// — Step 3: Read from source, write to dest —
while ((bytes_read = read(src, buffer, sizeof(buffer))) > 0) {
    write(dst, buffer, bytes_read);
}

// — Step 4: Close both files —————
close(src);
close(dst);

printf("File copied successfully: '%s' → '%s'\n", argv[1], argv[2]);

return 0;
}

```

Compile & Run

bash

```
gcc copy.c -o copy
./copy input.txt output.txt
```

Error Handling — What We Check & Why

Check

Why

```
argc != 3
```

User must give exactly 2 file paths

The Rule

c

argc = total number of arguments INCLUDING the program name itself

So when you run:

```
bash

./copy input.txt output.txt
...

| argv[0] | argv[1] | argv[2] |
|---|---|---|
| `./copy` | `input.txt` | `output.txt` |
...

argc = 3   (program name + 2 files)
```

That's why we check `argc != 3` 



That's why we check `argc != 3` 

Now Compare With `stat`

```
bash

./stat 10$20$30
...

| argv[0] | argv[1] |
|---|---|
| `./stat` | `10$20$30` |
...

argc = 2   (program name + 1 argument)
...

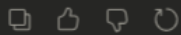
That's why we checked `argc < 2` there 

---

# The Simple Formula
...

argc = 1 + number of arguments you pass
```

✂️ `argv[0]` is always the program name — it's always counted in `argc`. That's the only thing to remember! 😊



reading, and writing operations.

4. Write a program that lists all files and directories in the current directory using system calls. The program should traverse the directory structure recursively and print the names of all files and directories found, along with their respective types (file or directory).

🧠 New Concepts First

New System Calls: `opendir()`, `readdir()`, `closedir()`

Think of a directory like a book:

```
opendir() = open the book
readdir() = read one entry at a time (one page at a time)
closedir() = close the book
```

New Header Needed:

```
c
#include <dirent.h> // for opendir, readdir, closedir
#include <sys/stat.h> // for stat(), checking file type
```

`opendir()` — Open a directory

```
c
DIR *dir = opendir("."); // "." means current directory
if (dir == NULL) {
    printf("Cannot open directory!\n");
    exit(1);
}
```

`DIR*` is like `FILE*` but for directories

`readdir()` — Read one entry

```
c
struct dirent *entry = readdir(dir);
```

`struct dirent` gives you:

Field	What it contains
<code>entry->d_name</code>	Name of file/directory
<code>entry->d_type</code>	Type — file or directory



Checking Type with `d_type`

```
c
if (entry->d_type == DT_DIR) {
    // it's a directory
}
if (entry->d_type == DT_REG) {
    // it's a regular file
}
...
---
```

🧠 Recursion — The Key Concept Here

"Traverse recursively" means:

...
List current folder
→ if you find a subfolder → go inside it → list that too
→ if you find a file → print it
→ come back → continue listing
...

Like exploring a building:
...

Floor 1 → Room 1 → check inside → back to Floor 1 → Room 2 ...

⚠ Skip `.` and `..`

Every directory has two special entries:

- `.` = current directory itself
- `..` = parent directory

If we don't skip them → infinite loop! ❌

```
c
if (strcmp(entry->d_name, ".") == 0 ||
    strcmp(entry->d_name, "..") == 0) {
    continue; // skip these
}
```

✅ Complete Program

```
c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h> // opendir, readdir, closedir
#include <sys/stat.h> // stat()
#include <sys/types.h>

// — Recursive function —————
// path = which directory to list
// depth = how deep we are (for indentation)
void listDirectory(const char *path, int depth) {

    DIR *dir;
    struct dirent *entry;

    // — Open the directory —————
    dir = opendir(path);
    if (dir == NULL) {
        printf("Cannot open directory: %s\n", path);
        return;
    }

    // — Read entries one by one —————
    while ((entry = readdir(dir)) != NULL) {
```



```

// — Read entries one by one —————
while ((entry = readdir(dir)) != NULL) {

    // Skip . and .. to avoid infinite loop
    if (strcmp(entry->d_name, ".") == 0 ||
        strcmp(entry->d_name, "..") == 0) {
        continue;
    }

    // — Build full path —————
    // e.g. "./myFolder/file.txt"
    char fullPath[1024];
    snprintf(fullPath, sizeof(fullPath), "%s/%s", path, entry->d_name);

    // — Print with indentation —————
    // depth controls how many spaces to print
    for (int i = 0; i < depth; i++) printf("    ");

    if (entry->d_type == DT_DIR) {
        // It's a DIRECTORY — print and recurse into it
        printf("[DIR]  %s\n", entry->d_name);
        listDirectory(fullPath, depth + 1); // go deeper!
    }
    else {
        // It's a FILE
        printf("[FILE] %s\n", entry->d_name);
    }
}

```

```

    }
    else {
        // It's a FILE
        printf("[FILE] %s\n", entry->d_name);
    }
}

// — Close the directory —————
closedir(dir);
}

int main() {

    printf("Listing all files and directories:\n\n");
    listDirectory(".", 0); // start from current dir, depth 0
    return 0;
}

```

▶ Compile & Run

```
bash

gcc listdir.c -o listdir
./listdir
...

---

# Expected Output
...

Listing all files and directories:

[DIR]  myFolder
      [FILE] file1.txt
      [FILE] file2.txt
      [DIR]  subFolder
            [FILE] deep.txt
[FILE] listdir.c
[FILE] listdir
[FILE] copy.c
...
```

The Core Reason

`execvp("ls", "ls", NULL)` lists files BUT:

`execvp` REPLACES your process entirely

- you get a flat list printed to screen
- you CANNOT do anything with the results in your code
- recursion is IMPOSSIBLE ❌

With `opendir/readdir`:

your program stays in control

- you can CHECK each entry's type yourself
- you can DECIDE to recurse into folders
- you can FORMAT output however you want ✅

Think Of It This Way

Approach	Like...
<code>execvp("ls")</code>	Asking someone else to read a book for you — you just hear the result
<code>opendir/readdir</code>	Reading the book yourself — you can stop, think, go back, take notes

Applied To This Question

"lists ALL files and directories... *traverse recursively*... *print their types*"

"traverse recursively" → need to go into subfolders ourselves → opendir/readdir



"print their types" → need to CHECK each entry's type → d_type



"using system calls" → explicitly told to use system calls → opendir/readdir



`execlp("ls")` fails all 3 checks ❌

When To Use What — Simple Rule

Just run a command, don't care about results

→ `fork()` + `execlp()`

Need to work WITH the results inside your code

→ use the actual system calls directly

✳ Your exam question will always hint which to use — words like "recursively", "check type", "process each file" = system calls directly. Words like "execute a command" = `execlp`. 😊

🧠 First — The Big Picture

`main()` calls `listDirectory(".", 0)`

→ opens current folder

→ reads each item one by one

→ if item is FILE → print it

→ if item is DIR → print it + go inside it (recursion)

→ close folder

📦 The Headers

```
c
#include <dirent.h> // gives us DIR, struct dirent, opendir, readdir, closedir
#include <sys/stat.h> // gives us DT_DIR, DT_REG
#include <sys/types.h> // gives us data types
```

Just like `#include <unistd.h>` gave us `fork()`, these headers give us directory tools.

The Function Signature

```
c
void listDirectory(const char *path, int depth)
```

Part	Meaning
<code>void</code>	returns nothing
<code>const char *path</code>	which folder to open — e.g. <code>"."</code> or <code>"./myFolder"</code>
<code>int depth</code>	how deep we are — controls indentation

When called first time:

```
c
listDirectory(".", 0)
// path = "." → current directory
// depth = 0 → no indentation yet
```

Declaring Variables

```
c
DIR *dir;           // like FILE* but for directories
struct dirent *entry; // represents ONE item inside the folder
...
```

Think of it like:
...

`DIR*` = the folder *itself* (the container)
`struct dirent` = one item inside that folder

Opening The Directory

```
c
dir = opendir(path);
if (dir == NULL) {
    printf("Cannot open directory: %s\n", path);
    return;
}
...
...

opendir(".") → opens current folder
              → returns DIR* pointer if success
              → returns NULL if failed

if NULL → print error and return (stop this function)
```

The While Loop — Reading Entries

c

```
while ((entry = readdir(dir)) != NULL) {  
    ...  
}
```

Break this down:

...

```
readdir(dir)    → reads ONE item from folder, moves to next  
entry =         → stores that item in entry  
!= NULL         → keep looping until no more items  
...
```

Like reading a book:

...

```
readdir = turn one page  
entry   = what's on that page  
NULL    = no more pages → stop
```

Skipping . and ..

c

```
if (strcmp(entry->d_name, ".") == 0 ||  
    strcmp(entry->d_name, "..") == 0) {  
    continue;  
}  
...
```

Every folder has 2 hidden entries:

...

```
.    = this folder itself  
..   = parent folder  
...
```

`strcmp` compares two strings:

...

```
strcmp("hello", "hello") → returns 0 (they match)  
strcmp("hi",      "hello") → returns non-zero (different)  
...
```

So:

...

```
if entry name is "." → skip it (continue = go to next loop)  
if entry name is ".." → skip it
```

Without this → program enters  → enters  again → infinite loop ❌

📁 Building The Full Path

```
C
char fullPath[1024];
snprintf(fullPath, sizeof(fullPath), "%s/%s", path, entry->d_name);
...
```

We need the ****complete path**** to pass into recursion:

```
...
path      = "."
entry->d_name = "myFolder"
fullPath   = "./myFolder"
...
```

Another example:

```
...
path      = "./myFolder"
entry->d_name = "file.txt"
fullPath   = "./myFolder/file.txt"
...
```

`snprintf` = safe version of `printf` that writes into a string instead of screen.

📁 Printing Indentation

```
C
for (int i = 0; i < depth; i++) printf("    ");
...
```

`depth` controls how many times we print 4 spaces:

```
...
depth = 0 → no spaces    → root level
depth = 1 → 4 spaces     → inside 1 folder
depth = 2 → 8 spaces     → inside folder inside folder
...
```

Output looks like:

```
...
[DIR] myFolder      ← depth 0, no spaces
  [FILE] file1.txt  ← depth 1, 4 spaces
  [DIR] subFolder   ← depth 1, 4 spaces
    [FILE] deep.txt ← depth 2, 8 spaces
...
```

Checking Type & Recursing

```
c
if (entry->d_type == DT_DIR) {
    printf("[DIR]  %s\n", entry->d_name);
    listDirectory(fullPath, depth + 1); // go deeper!
}
else {
    printf("[FILE] %s\n", entry->d_name);
}
...
...

DT_DIR → it's a directory
        → print [DIR] name
        → call listDirectory AGAIN with:
            fullPath = go into this folder
            depth + 1 = one level deeper (more indentation)

DT_REG → it's a regular file
        → just print [FILE] name
        → no recursion needed
```

Closing The Directory

```
c
closedir(dir);
...
```

Always close when done – same as `close(fd)` for files. Frees resources.

Line	What it does
<code>DIR *dir</code>	Container for the open folder
<code>struct dirent *entry</code>	Represents one item in folder
<code>opendir(path)</code>	Opens the folder
<code>readdir(dir)</code>	Reads next item in folder
<code>entry->d_name</code>	Name of current item
<code>strcmp(...) == 0</code>	Skip <code>.</code> and <code>..</code>
<code>snprintf(fullPath...)</code>	Build complete path for recursion
<code>for(depth) printf spaces</code>	Indent based on how deep we are
<code>DT_DIR</code>	This item is a directory
<code>listDirectory(fullPath, depth+1)</code>	Go inside that directory
<code>DT_REG</code>	This item is a file
<code>closedir(dir)</code>	Close folder when done

IPC

ORDINARY PIPES

🧠 FIRST — What is IPC?

So far we know `fork()` creates two processes. But here's the problem:

Parent process has its OWN memory

Child process has its OWN memory

They CANNOT see each other's variables!

So how do they talk to each other? That's exactly what IPC solves.

IPC = ways for processes to communicate with each other

Your lab has 3 IPC methods:

Method	Like...
Ordinary Pipes	A water pipe — one end writes, other end reads
Named Pipes	Same but has a name and lives as a file
Shared Memory	A whiteboard both can read and write

🔵 EXPERIMENT 6a: Ordinary Pipes

🧠 Concept — What is a Pipe?

Imagine a physical pipe:

Parent (writer) ———pipe——> Child (reader)
WRITE END READ END

- One side **writes** data in
- Other side **reads** data out
- Only works **one way** (unidirectional)
- Only works between **related processes** (parent-child)

🧠 New Concepts Before Code

Concept 1: `pipe()` — Create a pipe

```
c
int fd[2];    // fd = file descriptors, array of 2
pipe(fd);     // creates the pipe
...
```

After `'pipe(fd)'`:

```
...
fd[0] = READ  END  → child reads from here
fd[1] = WRITE END  → parent writes from here
...
```

Think of `'fd'` as two holes in the pipe:

```
...
fd[1] ==> fd[0]
WRITE END          READ END
(parent puts data in) (child takes data out)
...
```

Concept 2: Why Close Unused Ends?

This is confusing at first. Here's why:

After `'fork()'`, **both** parent and child have **both** ends of the pipe. But:

```
...
Parent should ONLY write → close its READ  end (fd[0])
Child  should ONLY read  → close its WRITE end (fd[1])
...
```

If you don't close unused ends:

- Child never gets EOF → waits forever → ****hangs**** ❌

```
...
PARENT:  close(fd[0]) → keep fd[1] → write into pipe
CHILD:   close(fd[1]) → keep fd[0] → read from pipe
```

Concept 3: `#define` — Constants

```
c
#define BUFFER_SIZE 25
#define READ_END 0
#define WRITE_END 1
```

These are just named constants — makes code readable:

```
c
fd[READ_END] // same as fd[0] but clearer!
fd[WRITE_END] // same as fd[1] but clearer!
```

1. Create constants `READ_END = 0`, `WRITE_END = 1` and the size of the pipe

```
c
int main(void) {
    char write_msg[BUFFER_SIZE] = "Greetings";
    char read_msg[BUFFER_SIZE];
    int fd[2];
    pid_t pid;
```

`write_msg` = what parent will send `read_msg` = empty buffer where child will store what it reads `fd[2]` = the two pipe ends `pid` = to store fork result

```
c
if (pipe(fd) == -1) {
    fprintf(stderr, "Pipe failed");
    return 1;
}
```

Create the pipe. If it fails (`-1`) → print error and exit. `fprintf(stderr, ...)` = print error to error screen (same as `printf` but for errors)

```
c
pid = fork();
if (pid < 0) {
    fprintf(stderr, "Fork Failed");
    return 1;
}
```

Create child process. If fork fails → exit.

🧠 Final idea

- `write()` = send data
- `fd[1]` = where to send (pipe write end)
- `write_msg` = what to send
- `strlen + 1` = send full string including `\0`

◆ Easy way to remember 🧠

`write(fd, buffer, count)`

- `fd` → WHERE
- `buffer` → WHAT
- `count` → HOW MUCH

Just reverse of write:

↗ C

```
read(fd, buffer, count);
```

- reads from `fd` into `buffer`

C

```
if (pid > 0) { /* parent process */
    close(fd[READ_END]);
    write(fd[WRITE_END], write_msg, strlen(write_msg) + 1);
    close(fd[WRITE_END]);
}
...
```

Step by step:

...

<code>close(fd[READ_END])</code>	→ parent doesn't need to read, close it
<code>write(fd[WRITE_END],</code>	→ write INTO the pipe
<code>write_msg,</code>	→ what to write ("Greetings")
<code>strlen+1)</code>	→ how many bytes (+1 for <code>\0</code> end of string)
<code>close(fd[WRITE_END])</code>	→ done writing, close write end too

```
c
else { /* child process */
    close(fd[WRITE_END]);
    read(fd[READ_END], read_msg, BUFFER_SIZE);
    printf("read %s\n", read_msg);
    close(fd[READ_END]);
}
...

Step by step:
...

close(fd[WRITE_END]) → child doesn't need to write, close it
read(fd[READ_END],    → read FROM the pipe
    read_msg,          → store data here
    BUFFER_SIZE)       → read up to 25 bytes
printf(read_msg)       → print what we got → "Greetings"
close(fd[READ_END])   → done reading, close it
...
```

✓ In-Lab Task — Take `write_msg` from User

The lab says: "Modify code so `write_msg` comes from user terminal"

Just change this one part in parent:

```
c
if (pid > 0) {
    close(fd[READ_END]);

    // OLD:
    // write(fd[WRITE_END], write_msg, strlen(write_msg) + 1);

    // NEW - take from user:
    printf("Enter message: ");
    fgets(write_msg, BUFFER_SIZE, stdin); // read from keyboard
    write(fd[WRITE_END], write_msg, strlen(write_msg) + 1);

    close(fd[WRITE_END]);
}
...
```

◆ The code:

```
</> C
printf("Enter message: ");
fgets(write_msg, BUFFER_SIZE, stdin);
```

◆ What does this do?

👉 It asks the user to type a message from the keyboard and stores it in `write_msg`.

🔍 Line-by-line

1 `printf(...)`

```
</> C
printf("Enter message: ");
```

👉 Just displays:

```
Enter message:
```



Structure:

```
</> C
fgets(buffer, size, source);
```

2 `fgets(...)`

```
</> C
fgets(write_msg, BUFFER_SIZE, stdin);
```

🔍 Breakdown:

✓ `write_msg`

👉 The array (buffer) where input will be stored

Example:

```
</> C
char write_msg[100];
```

Creating an Ordinary Pipe:

```
1  #include <sys/types.h>
2  #include <stdio.h>
3  #include <string.h>
4  #include <unistd.h>
5  #define BUFFER_SIZE 25
6  #define READ_END 0
7  #define WRITE_END 1
8  int main(void) {
9      char write_msg[BUFFER_SIZE] = "Greetings";
10     char read_msg[BUFFER_SIZE];
11     int fd[2];
12     pid_t pid;
13
14     if (pipe(fd) == -1) { // create a pipe
15         fprintf(stderr, "Pipe failed");
16         return 1;
17     }
18     pid = fork();
19     if (pid < 0) { /* error occurred */
20         fprintf(stderr, "Fork Failed");
21         return 1;
22     }
23     if (pid > 0) { /* parent process */
24         close(fd[READ_END]);
25         write(fd[WRITE_END], write_msg, strlen(write_msg) + 1);
26         close(fd[WRITE_END]);
27     }
28     else { /* child process */
29         close(fd[WRITE_END]); // close the unused end of the pipe
30         read(fd[READ_END], read_msg, BUFFER_SIZE); // read from the pipe
31         printf("read %s", read_msg);
32         close(fd[READ_END]); // close the read end of the pipe
33     }
34     return 0;
35 }
```

NAMED PIPES

c



```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <string.h>

#define FIFO_FILE "/tmp/myfifo"

int main() {

    int fd;
    char buffer[BUFSIZ];    // BUFSIZ is predefined in stdio.h = 8192 bytes
    ssize_t num_bytes;      // ssize_t = signed version of size_t, for bytes read/wr

    // — Create the named pipe —————
    mkfifo(FIFO_FILE, 0666);

    // — Open named pipe for writing —————
    fd = open(FIFO_FILE, O_WRONLY);
    if (fd == -1) {
        perror("open");
        exit(EXIT_FAILURE);
    }

    // — Producer loop —————
    while (1) {
        printf("Producer: Enter a message (or 'exit' to quit): ");
        fgets(buffer, BUFSIZ, stdin);

        // Write message to named pipe
        num_bytes = write(fd, buffer, strlen(buffer));
        if (num_bytes == -1) {
            perror("write");
            exit(EXIT_FAILURE);
        }

        // Check if user typed "exit"
        if (strncmp(buffer, "exit", 4) == 0) {
            break;
        }
    }
}
```



```

    }

    // — Cleanup —————
    close(fd);
    unlink(FIFO_FILE); // delete the named pipe file

    return 0;
}

```

Compile & run:

```

bash

gcc producer.c -o producer
./producer

```

⚠ Run consumer in second terminal **before** typing messages! Otherwise pipe has no reader and program hangs. 😊

Here's the complete consumer code! 🍷

```

c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <string.h>

#define FIFO_FILE "/tmp/myfifo"

int main() {

    int fd;
    char buffer[BUFSIZ];
    ssize_t num_bytes;

    // — Open named pipe for reading —————
    fd = open(FIFO_FILE, O_RDONLY);
    if (fd == -1) {
        perror("open");
        exit(EXIT_FAILURE);
    }
}

```

```
// — Open named pipe for reading —————
fd = open(FIFO_FILE, O_RDONLY);
if (fd == -1) {
    perror("open");
    exit(EXIT_FAILURE);
}

// — Keep reading until producer closes pipe —
while ((num_bytes = read(fd, buffer, BUFSIZ)) > 0) {
    printf("Consumer: Received message: %s", buffer);
}

// — Cleanup —————
close(fd);

return 0;
}
```

How to Run Both Together

```
bash

# Terminal 1 — start consumer FIRST
gcc consumer.c -o consumer
./consumer

# Terminal 2 — then run producer
gcc producer.c -o producer
./producer
```

📌 Consumer starts **first** because `open()` on a named pipe **blocks** (waits) until the other end is also opened. So consumer waits → producer opens → both connect → communication begins!



Key Difference Between Producer & Consumer

	Producer	Consumer
<code>mkfifo()</code>	✅ Creates the pipe	❌ Not needed — pipe already exists
<code>open()</code> flag	<code>O_WRONLY</code>	<code>O_RDONLY</code>
Main job	<code>write()</code> to pipe	<code>read()</code> from pipe
<code>unlink()</code>	✅ Deletes pipe at end	❌ Not needed

EXPERIMENT 6b: Named Pipes (FIFO)

Concept — What's Different From Ordinary Pipes?

Feature	Ordinary Pipe	Named Pipe (FIFO)
Lives as a file?	✗ No	✓ Yes
Works between unrelated processes?	✗ No	✓ Yes
Needs <code>fork()</code> ?	✓ Yes	✗ No
How long does it live?	Dies with process	Lives until deleted

What is a Named Pipe?

A Named Pipe (FIFO) is a **special file** created on disk that allows communication between processes.

👉 It behaves like a pipe, but:

- It has a **name**
- It exists in the **file system**
- Different programs can use it

Basic Idea

Producer (Program 1) → /tmp/myfifo → Consumer (Program 2)
writes (FIFO) reads

- One program writes data into the FIFO
- Another program reads from it
- They can be **completely separate programs**

How to Create a Named Pipe

```
</> Bash
mkfifo /tmp/myfifo
```

👉 This creates a special file:

```
</> Bash
ls -l /tmp/myfifo
```

Output:

```
prw-r--r-- 1 user user 0 Mar 29 12:00 /tmp/myfifo
```

- **p** at the start = pipe (FIFO)

◆ Simple Example

● Terminal 1 (Writer)

```
</> Bash
echo "Hello from producer" > /tmp/myfifo
```

● Terminal 2 (Reader)

```
</> Bash
cat < /tmp/myfifo
```

Output:

```
Hello from producer
```

🧠 Final Understanding

Named Pipe = a **file-based communication channel** between separate processes

🧠 New Calls

Call	Meaning
<code>mkfifo(path, mode)</code>	Create the named pipe file
<code>open(path, O_WRONLY)</code>	Producer opens it for writing
<code>open(path, O_RDONLY)</code>	Consumer opens it for reading
<code>unlink(path)</code>	Delete the named pipe when done
<code>fgets(buf, size, stdin)</code>	Read line from keyboard

🔍 Producer Code — Line by Line

```
c
#define FIFO_FILE "/tmp/myfifo"
```

The name/path of our named pipe file.

```
c
mkfifo(FIFO_FILE, 0666);
```

Creates the named pipe as a file at `/tmp/myfifo` `0666` = everyone can read and write

write

```
c
fd = open(FIFO_FILE, O_WRONLY);
if (fd == -1) {
    perror("open");
    exit(EXIT_FAILURE);
}
```

Open the pipe for writing only. `perror()` = prints the exact system error message.

```
c
while (1) {
    printf("Producer: Enter a message (or 'exit' to quit): ");
    fgets(buffer, BUFSIZ, stdin);
}
```

Infinite loop — keep asking user for messages. `fgets` reads one line from keyboard into `buffer`.

```
c
num_bytes = write(fd, buffer, strlen(buffer));
if (num_bytes == -1) {
    perror("write");
    exit(EXIT_FAILURE);
}
```

Write whatever user typed into the named pipe. ↓

Write whatever user typed into the named pipe.

```
c
if (strncmp(buffer, "exit", 4) == 0) {
    break;
}
```

`strncmp` compares first 4 characters of buffer with `"exit"`. If user typed "exit" → break out of loop.

`strcmp` compares first 4 characters of buffer with `"exit"`. If user typed `"exit"` → break out of loop.

```
c
close(fd);
unlink(FIFO_FILE);
```

Close the pipe. `unlink` = delete the named pipe file from filesystem.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <string.h>

#define FIFO_FILE "/tmp/myfifo"

int main() {
    int fd;
    char buffer[BUFSIZ];
    ssize_t num_bytes;

    // Open named pipe for READING
    fd = open(FIFO_FILE, O_RDONLY);
    if (fd == -1) {
        perror("open");
        exit(EXIT_FAILURE);
    }

    // Keep reading until producer closes pipe
    while ((num_bytes = read(fd, buffer, BUFSIZ)) > 0) {
        printf("Consumer: Received message: %s", buffer);
    }

    close(fd);
    return 0;
}
```

🔴 How to Run Producer + Consumer

They run in two separate terminals:

```
bash

# Terminal 1 - run producer first
gcc producer.c -o producer
./producer

# Terminal 2 - run consumer
gcc consumer.c -o consumer
./consumer
...
...

Producer types: "Hello!"    → Consumer prints: "Hello!"
Producer types: "exit"      → both programs stop
...
```

Shared Pipe

```
c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    const int SIZE = 4096;
    const char *name = "OS";
    const char *message_0 = "Hello";
    const char *message_1 = "World!";

    int fd;
    char *ptr;
    pid_t pid;
```

```
// — Step 1: Create shared memory object ————
fd = shm_open(name, O_CREAT | O_RDWR, 0666);
if (fd == -1) {
    perror("shm_open");
    exit(EXIT_FAILURE);
}

// — Step 2: Set size of shared memory ————
ftruncate(fd, SIZE);
```

```

// — Step 3: Map shared memory into address space
ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
if (ptr == MAP_FAILED) {
    perror("mmap");
    exit(EXIT_FAILURE);
}

// — Step 4: Write messages to shared memory —
sprintf(ptr, "%s", message_0); // write "Hello"
ptr += strlen(message_0);      // move pointer forward
sprintf(ptr, "%s", message_1); // write "World!"
ptr += strlen(message_1);      // move pointer forward

printf("Producer: Written to shared memory: HelloWorld!\n");

```

```

// — Step 5: Fork — child runs consumer —
pid = fork();
if (pid < 0) {
    perror("fork");
    exit(EXIT_FAILURE);
}
if (pid == 0) {
    // CHILD — launch consumer program
    execlp("./consumer", "consumer", NULL);
    // only reaches here if execlp failed
    perror("execlp");
    exit(EXIT_FAILURE);
}

// — Step 6: Parent waits for consumer to finish
wait(NULL);
printf("Producer: Consumer finished. Cleaning up!\n");

// — Step 7: Cleanup — AFTER consumer is done —
shm_unlink(name);

return 0;
}

```

We open existing shared memory by `shm_open(name, O_CREAT | O_RDWR, 0666);`

THEN WE truncate size of shared memory

`ftruncate(fd, SIZE);`

Then we map shared memory into address space by


```
Ptr = (char *)mmap(0 , SIZE , PROT_READ | PROT_WRITE, MAP_SHARED , fd , 0);
```

Then we write messages to shared memory

Sprint

Then we do fork for running child

After running `execlp` in child , we do `wait(NULL)` and then do `shm_unlink(name)`

Consumer Code (unchanged)

```
c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main() {

    const int SIZE = 4096;
    const char *name = "OS";

    int fd;
    char *ptr;

    // — Open existing shared memory —————
    fd = shm_open(name, O_RDONLY, 0666);
    if (fd == -1) {
        perror("shm_open");
        exit(EXIT_FAILURE);
    }

    // — Map into address space —————
    ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap");
        exit(EXIT_FAILURE);
    }

    // — Read and print from shared memory —————
    printf("Consumer: Received: %s\n", (char *)ptr);

    // — Cleanup —————
    shm_unlink(name);

    return 0;
}
```

How to Compile & Run

```
bash

# Compile both - -lrt is REQUIRED for shm_open
gcc producer.c -o producer -lrt
gcc consumer.c -o consumer -lrt

# Run ONLY producer - it launches consumer itself!
./producer
...

---

# Expected Output
...

Producer: Written to shared memory: HelloWorld!
Consumer: Received: HelloWorld!
Producer: Consumer finished. Cleaning up!
...
```

◆ Key Differences

1 Communication Style

● Named Pipe

Process A → FIFO → Process B



- One writes, one reads
- Like a **pipe/queue**

● Shared Memory

Process A ↔ Shared Memory ↔ Process B



- Both can **read & write directly**
- Like a **common whiteboard**

🧠 New Calls

Call	Meaning
<code>shm_open(name, flags, mode)</code>	Create/open shared memory object
<code>ftruncate(fd, size)</code>	Set the size of shared memory
<code>mmap(...)</code>	Map shared memory into your program's address space
<code>shm_unlink(name)</code>	Delete shared memory when done

🧠 What is `mmap()`?

This is the trickiest new call. Simply:

`mmap()` = "give me a pointer to the shared memory so I can use it like a normal char array"

```
c
ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
```

Part	Meaning
<code>0</code>	Let OS choose where in memory
<code>SIZE</code>	How many bytes (4096)
<code>PROT_READ PROT_WRITE</code>	Can read AND write
<code>MAP_SHARED</code>	Share with other processes
<code>fd</code>	The shared memory file descriptor
<code>0</code>	Start from beginning

After this → `ptr` is like a `char[]` array but in shared memory!



🧠 What is `mmap()` ?

This is the trickiest new call. Simply:

`mmap()` = "give me a pointer to the shared memory so I can use it like a normal char array"

c

```
ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
```

Part	Meaning
0	Let OS choose where in memory
SIZE	How many bytes (4096)
PROT_READ PROT_WRITE	Can read AND write
MAP_SHARED	Share with other processes
fd	The shared memory file descriptor
0	Start from beginning

After this → `ptr` is like a `char[]` array but in  red memory!

🔗 Producer Code — Line by Line

c

```
const int SIZE = 4096;           // shared memory size in bytes
const char *name = "OS";         // name of shared memory object
const char *message_0 = "Hello";
const char *message_1 = "World!";
int fd;
char *ptr;
```

Variables. `ptr` will point to shared memory.

c

```
fd = shm_open(name, O_CREAT | O_RDWR, 0666);
```

Create shared memory named "OS". `O_CREAT` = create it, `O_RDWR` = read+write, `0666` = permissions.

`0666` = permissions.

```
c
ftruncate(fd, SIZE);
```

Set size of shared memory to 4096 bytes.
Like "reserving" that much space.

```
c
ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
```

Get pointer to shared memory — now `ptr` points to it!

Get pointer to shared memory — now `ptr` points to it!

```
c
sprintf(ptr, "%s", message_0); // write "Hello" at ptr
ptr += strlen(message_0);      // move ptr forward past "Hello"
sprintf(ptr, "%s", message_1); // write "World!" after it
ptr += strlen(message_1);      // move ptr forward again
```

Write both messages into shared memory one after another. `sprintf` = like `printf` but writes INTO a string/pointer.

We need to declare size , name , mesg 0 , msg 1 , fd and *ptr

Consumer Code — Line by Line

```
c
fd = shm_open(name, O_RDONLY, 0666);
```

Open the same shared memory (name = `"OS"`) — read only this time.

```
c
ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
```

Get pointer to same shared memory block.

```
c
printf("%s", (char *)ptr);
```

Read and print from shared memory → prints `"HelloWorld!"`

```
c
shm_unlink(name);
...
> Delete the shared memory object — cleanup.
```

⚠️ Why Does Consumer Give Runtime Error?

The lab asks: *"Consumer gives runtime error – why?"*

...

Producer creates shared memory → writes to it → exits → shared memory GONE

Consumer runs AFTER → tries to open shared memory → IT DOESN'T EXIST! ❌

Fix = Run them concurrently using `fork()` in producer:

```
c
// In producer, after writing to shared memory:
pid_t pid = fork();
if (pid == 0) {
    // child runs consumer
    execlp("./consumer", "consumer", NULL);
    exit(0);
}
wait(NULL);
shm_unlink(name); // only unlink AFTER consumer is done!
...
```

🧠 Master Summary – All 3 IPC Methods

...

	Ordinary Pipe	Named Pipe	Shared Memory
Create	<code>pipe(fd)</code>	<code>mkfifo()</code>	<code>shm_open()</code>
Write	<code>write(fd[1]...)</code>	<code>write(fd...)</code>	<code>sprintf(ptr...)</code>
Read	<code>read(fd[0]...)</code>	<code>read(fd...)</code>	<code>printf(ptr)</code>
Cleanup	<code>close(fd)</code>	<code>close()+unlink()</code>	<code>shm_unlink()</code>
Needs fork?	YES	NO	NO (but helpful)
Speed	Fast	Medium	Fastest

Compile Commands

```
bash

# Ordinary pipe
gcc pipe.c -o pipe

# Named pipe
gcc producer.c -o producer
gcc consumer.c -o consumer

# Shared memory - needs -lrt flag!
gcc producer_shm.c -o producer_shm -lrt
gcc consumer_shm.c -o consumer_shm -lrt
```

⚠ `-lrt` = link the real-time library — required for `shm_open()` to work!

EXAM TASKS

Shell scripting :

Task 1:

Keep project directories tidy.

Write a **Bash script** that:

1. Takes a directory path as input.
2. Deletes .log, .tmp, and .bak files after asking for confirmation.
3. Displays the number of files deleted and the remaining files in the directory.

Step 2 — Paste the entire script

```
bash

#!/bin/bash

read -p "Enter directory path: " dir

if [ ! -d "$dir" ]; then
    echo "Error: Directory does not exist!"
    exit 1
fi

files=$(find "$dir" -type f \( -name "*.log" -o -name "*.tmp" -o -name "*.bak" \))

if [ -z "$files" ]; then
    echo "No .log, .tmp or .bak files found!"
    exit 0
fi

echo "Following files will be deleted:"
echo "$files"
echo ""
read -p "Are you sure you want to delete? (yes/no): " confirm
```

```
fi

echo "Following files will be deleted:"
echo "$files"
echo ""
read -p "Are you sure you want to delete? (yes/no): " confirm

if [ "$confirm" = "yes" ]; then
    count=0
    for file in $files; do
        rm "$file"
        echo "Deleted: $file"
        count=$((count + 1))
    done
    echo ""
    echo "Total files deleted: $count"
else
    echo "Deletion cancelled!"
fi

echo ""
echo "Remaining files in $dir:"
ls "$dir"
EOF
```


Task 2:

A system administrator wants to track who logs into the system.

Write a **Bash script** that:

1. Takes a filename as input.
2. Stores the list of currently logged-in users in the file.
3. Adds the current date and time at the top of the file.
4. Displays the saved information on the terminal.

Implement the script in Bash, run it, and display the terminal output.

Type This:

```
bash

#!/bin/bash

# Step 1: Take filename as input
read -p "Enter filename: " filename

# Step 2: Write current date/time at TOP of file
echo "Date and Time: $(date)" > "$filename"

# Step 3: Append logged-in users after date
echo "Logged-in Users:" >> "$filename"
who >> "$filename"

# Step 4: Display the file contents
echo ""
echo "Saved information:"
cat "$filename"
```

Task 3:

The system contains many empty directories that need to be cleaned.

Write a **Bash script** that:

1. Takes a directory path as input.
2. Finds all empty folders inside it.
3. Deletes those empty folders.
4. Displays the number of folders removed.

Implement the script in Bash, run it on a sample directory, and display the terminal output.

Got it! Here it is:

```
bash

#!/bin/bash

read -p "Enter directory path: " dir

if [ ! -d "$dir" ]; then
    echo "Error: Directory does not exist!"
    exit 1
fi

count=$(find "$dir" -type d -empty | wc -l)

if [ "$count" -eq 0 ]; then
    echo "No empty folders found!"
    exit 0
fi

find "$dir" -type d -empty -delete

echo "Total empty folders removed: $count"
echo ""
echo "Remaining contents in $dir:"
ls "$dir"
```

IPC SHARED

IPC:

1. You are designing a banking application in which a parent and child process collaborate to analyze transaction records. The parent process collects the transaction amounts of 10 customers and stores them in shared memory. After creating a child process using `fork()`, the child reads the stored values and determines how many transactions exceed 10,000. Once the analysis is complete, the parent process displays the number of high-value transactions. Write a C program using `fork()` and shared memory to perform this transaction analysis.

🧠 What Goes In Shared Memory?

We need to store:

- 10 transaction amounts → `int` array
- 1 count result → `int`

So shared memory layout:

```
[ t1 ][ t2 ][ t3 ]...[ t10 ][ count ]
  ↑               ↑
parent writes these child writes this
```

🧠 The 6 Steps You Already Know

...

Step 1 → `shm_open()` create shared memory

Step 2 → `ftruncate()` set size

Step 3 → `mmap()` get pointer

Step 4 → parent fills `transactions[]`

Step 5 → `fork()`
 child → reads `transactions[]`
 → counts > 10000
 → stores in `highCount`
 → `exit(0)`

Step 6 → `wait(NULL)`
 → print `highCount`
 → `shm_unlink()`



Producer (with fork — no crash)

```
c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

struct SharedData {
    int transactions[10];
    int highCount;
};

int main() {

    const int SIZE = sizeof(struct SharedData);
    const char *name = "BankApp";

    int fd;
    struct SharedData *ptr;
    pid_t pid;
```

```

// — Step 1: Create shared memory —————
fd = shm_open(name, O_CREAT | O_RDWR, 0666);
if (fd == -1) {
    perror("shm_open");
    exit(EXIT_FAILURE);
}

// — Step 2: Set size —————
ftruncate(fd, SIZE);

// — Step 3: Map into address space —————
ptr = (struct SharedData *)mmap(0, SIZE,
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
if (ptr == MAP_FAILED) {
    perror("mmap");
    exit(EXIT_FAILURE);
}

// — Step 4: Parent fills transactions —————
printf("Producer: Enter 10 transaction amounts:\n");
for (int i = 0; i < 10; i++) {
    printf("Transaction %d: ", i + 1);
    scanf("%d", &ptr->transactions[i]);
}
ptr->highCount = 0;

printf("Producer: Transactions stored in shared memory!\n\n");

```

```

// — Step 5: Fork - child runs consumer —————
pid = fork();
if (pid < 0) {
    perror("fork");
    exit(EXIT_FAILURE);
}

if (pid == 0) {
    // CHILD - launch consumer
    execlp("./consumer", "consumer", NULL);
    perror("execlp");
    exit(EXIT_FAILURE);
}

// — Step 6: Parent waits for consumer —————
wait(NULL);

// — Step 7: Read result & print —————
printf("\nProducer: High-value transactions (>10000): %d\n",
        ptr->highCount);

// — Step 8: Cleanup AFTER consumer done —————
shm_unlink(name);

return 0;
}

```

Consumer (separate program)

```

c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <unistd.h>

struct SharedData {
    int transactions[10];
    int highCount;
};

int main() {

    const int SIZE = sizeof(struct SharedData);
    const char *name = "BankApp";

    int fd;
    struct SharedData *ptr;

```

```

// — Step 1: Open existing shared memory ———
fd = shm_open(name, O_RDWR, 0666);
if (fd == -1) {
    perror("shm_open");
    exit(EXIT_FAILURE);
}

// — Step 2: Map into address space ———
ptr = (struct SharedData *)mmap(0, SIZE,
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
if (ptr == MAP_FAILED) {
    perror("mmap");
    exit(EXIT_FAILURE);
}

// — Step 3: Count transactions > 10000 ———
printf("Consumer: Analyzing transactions...\n");

int count = 0;
for (int i = 0; i < 10; i++) {
    if (ptr->transactions[i] > 10000) {
        count++;
    }
}

// — Step 4: Store result in shared memory ———
ptr->highCount = count;

printf("Consumer: Analysis done! High-value count = %d\n", count);

return 0;
}

```

Compile & Run

```

bash

# Compile both
gcc producer.c -o producer -lrt
gcc consumer.c -o consumer -lrt

# Run ONLY producer – it launches consumer itself!
./producer
...

```

2. You are developing a vehicle monitoring application that analyzes fuel consumption data. The parent process collects fuel consumption values for 10 vehicles and stores them in shared memory. The child process analyzes the data and counts how many vehicles consume more than 12 liters per 100 km. Once the computation is complete, the parent process displays the number of high fuel consumption vehicles. Write a C program using `fork()` and shared memory to implement this monitoring system.

Producer

```
c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

struct SharedData {
    float fuel[10];           // fuel consumption for 10 vehicles
    int highCount;            // vehicles consuming more than 12L
};

int main() {

    const int SIZE = sizeof(struct SharedData);
    const char *name = "FuelApp";

    int fd;
    struct SharedData *ptr;
    pid_t pid;
```



```

// — Step 1: Create shared memory —————
fd = shm_open(name, O_CREAT | O_RDWR, 0666);
if (fd == -1) {
    perror("shm_open");
    exit(EXIT_FAILURE);
}

// — Step 2: Set size —————
ftruncate(fd, SIZE);

// — Step 3: Map into address space —————
ptr = (struct SharedData *)mmap(0, SIZE,
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
if (ptr == MAP_FAILED) {
    perror("mmap");
    exit(EXIT_FAILURE);
}

// — Step 4: Collect fuel data —————
printf("Producer: Enter fuel consumption for 10 vehicles (L/100km):\n");
for (int i = 0; i < 10; i++) {
    printf("Vehicle %d: ", i + 1);
    scanf("%f", &ptr->fuel[i]);
}
ptr->highCount = 0;

printf("\nProducer: Fuel data stored in shared memory!\n\n");

```

```
printf("\nProducer: Fuel data stored in shared memory!\n\n");

// — Step 5: Fork - child runs consumer —————
pid = fork();
if (pid < 0) {
    perror("fork");
    exit(EXIT_FAILURE);
}

if (pid == 0) {
    // CHILD - launch consumer
    execlp("./consumer", "consumer", NULL);
    perror("execlp");
    exit(EXIT_FAILURE);
}

// — Step 6: Wait for consumer to finish —————
wait(NULL);

// — Step 7: Print result —————
printf("\nProducer: Vehicles with high fuel consumption (>12L): %d\n",
        ptr->highCount);

// — Step 8: Cleanup —————
shm_unlink(name);

return 0;
}
```

Consumer

c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <unistd.h>

struct SharedData {
    float fuel[10];
    int highCount;
};

int main() {

    const int SIZE = sizeof(struct SharedData);
    const char *name = "FuelApp";

    int fd;
    struct SharedData *ptr;

    // — Step 1: Open existing shared memory —————
    fd = shm_open(name, O_RDWR, 0666);
    if (fd == -1) {
        perror("shm_open");
        exit(EXIT_FAILURE);
    }
```

```

// — Step 2: Map into address space ———
ptr = (struct SharedData *)mmap(0, SIZE,
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
if (ptr == MAP_FAILED) {
    perror("mmap");
    exit(EXIT_FAILURE);
}

// — Step 3: Count vehicles consuming > 12L ———
printf("Consumer: Analyzing fuel consumption data...\n");

int count = 0;
for (int i = 0; i < 10; i++) {
    if (ptr->fuel[i] > 12.0) {
        count++;
    }
}

// — Step 4: Store result in shared memory ———
ptr->highCount = count;

printf("Consumer: Analysis done! High consumption vehicles = %d\n",
    count);

return 0;
}

```

Compile & Run

```

bash

gcc producer.c -o producer -lrt
gcc consumer.c -o consumer -lrt

./producer
...

---
```

System Calls

System Calls

1. A small utility program is designed to display the contents of the current directory. The main program creates a helper process to perform the listing operation. The helper process displays its **process ID** and **parent process ID** before executing the system command to show directory contents. The main program waits for the operation to finish and then prints a completion message.

Here you go! 🍷

```
c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    pid_t pid;

    pid = fork();

    pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        // — HELPER (CHILD) process —————
        printf("Helper Process PID          = %d\n", getpid());
        printf("Helper Process Parent PID = %d\n", getppid());
        printf("\nDirectory contents:\n");
        execlp("ls", "ls", NULL);

        // only reaches here if execlp failed
        perror("execlp");
        exit(EXIT_FAILURE);
    }
    else {
        // — MAIN (PARENT) process —————
        wait(NULL);
        printf("\nMain process: Helper finished. Operation complete!\n");
    }

    return 0;
}
```

3. A monitoring application retrieves system status information using a background process. The helper process prints its **PID** and **PPID**, executes a system command to display system status, and terminates. The main program waits for the helper process and prints a final message.

Here you go! 🍷

```
c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    pid_t pid;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        // — HELPER (CHILD) process —————
        printf("Helper Process PID      = %d\n", getpid());
        printf("Helper Process Parent PID = %d\n", getppid());
        printf("\nSystem Status:\n");
        execlp("uptime", "uptime", NULL);

        // only reaches here if execlp failed
        perror("execlp");
        exit(EXIT_FAILURE);
    }
    else {
        // — MAIN (PARENT) process —————
        wait(NULL);
        printf("\nMain process: System status retrieved. Done!\n");
    }

    return 0;
}
```

4. A system program checks file permissions in the current directory using a helper process. The helper process first prints its **PID** and **PPID**, then executes a system command to display file details. The main program waits for the process to complete before exiting.

c



```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    pid_t pid;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        // — HELPER (CHILD) process —————
        printf("Helper Process PID          = %d\n", getpid());
        printf("Helper Process Parent PID = %d\n", getppid());
        printf("\nFile Permissions in Current Directory:\n");
        execlp("ls", "ls", "-l", NULL);

        // only reaches here if execlp failed
        perror("execlp");
        exit(EXIT_FAILURE);
    }
    else {
```



```
        exit(EXIT_FAILURE);
    }
    else {
        // — MAIN (PARENT) process —————
        wait(NULL);
        printf("\nMain process: File permissions checked. Done!\n");
    }

    return 0;
}
```

Compile & Run

```
bash

gcc fileperm.c -o fileperm
./fileperm
` ``
```